GNR638 Assignment 2 Report

Amit Pandey, Sakshi Pandey, Sharath H N

March 16, 2026

Contents

1 Introduction

1.1 Dataset: AID (Aerial Image Dataset)

- Dataset used: local image-folder version of AID [1] under `train_data/`.
- Total images used in this study: **6,993**.
- Number of classes: **30**.
- Image type: RGB aerial-scene images from Google Earth.
- Original image size: 600×600 pixels.
- Split used in this report: **80% training / 20% validation**, stratified, seed = 123.
- Final split sizes: **5,594 training** and **1,399 validation** images.
- Separate test split: **not used**; the 20% validation split served as the hold-out evaluation set for all reported model results.
- Validation usage: used for checkpoint selection and final reporting, but **not** for gradient updates or parameter optimization.
- Preprocessing for all models: resize to 224×224 and ImageNet normalization.
- Few-shot subsets: fixed subsets with $k \in \{5, 10, 20, 50\}$ samples per class drawn from the training split.

1.2 Pre-trained Models

Three ImageNet-pretrained convolutional backbones were selected, each representing a distinct architectural design philosophy:

- **ResNet50** [2]: A widely used residual network with 50 layers organized into four stages of bottleneck blocks. Skip connections enable training of very deep networks without degradation. ResNet50 serves as the strong and well-understood baseline in this study, with approximately 25.6 million parameters.
- **EfficientNet-B0** [3]: A compact network designed via neural architecture search and scaled uniformly in depth, width, and resolution using a compound coefficient. Its use of mobile inverted bottleneck blocks (MBConv) with squeeze-and-excitation attention makes it highly parameter-efficient. EfficientNet-B0 has approximately 5.3 million parameters, the smallest of the three.
- **ConvNeXt-Tiny** [4]: A purely convolutional architecture modernized by incorporating design elements inspired by Vision Transformers, including depthwise convolutions with large 7×7 kernels, LayerNorm, GELU activations, and an inverted bottleneck structure. ConvNeXt-Tiny achieves competitive accuracy while retaining the inductive biases of convolutional networks, with approximately 28.6 million parameters.

For all backbones, the original ImageNet classification head was replaced with a new fully connected layer mapping to 30 output classes. The backbone weights were either frozen (linear probe) or partially/fully unfrozen depending on the experimental scenario.

1.3 The `timm` Library

All three backbones were loaded using the `timm` (PyTorch Image Models) library [5], an open-source collection of state-of-the-art image models for PyTorch. `timm` provides:

- Standardized pretrained weight checkpoints for hundreds of architectures.
- A unified `create_model` API that returns a model with or without the final classification head (`num_classes=0` for feature extraction).

- Consistent preprocessing configurations per model, including recommended input resolution and normalization statistics.
- Utilities for inspecting and modifying model internals (ex., layer-wise parameter iteration for selective freezing).

Using `timm` ensured that differences in observed performance across models were attributable to architectural behavior and adaptation strategy rather than inconsistent implementations or non-standard weight initializations.

1.4 Experimental Overview

Five experimental scenarios were conducted to analyze transfer learning behavior comprehensively:

1. **Scenario 4.1: Linear Probe**: All backbone parameters are frozen; only the 30-class classification head is trained. This isolates the quality of the pretrained feature representation.
2. **Scenario 4.2: Selective Fine-Tuning**: The final stage of each backbone (at most 20% of backbone parameters) is unfrozen alongside the classification head, enabling task-specific feature adaptation.
3. **Scenario 4.3: Few-Shot Learning**: Linear probing is performed with limited labeled data ($k = 5, 10, 20, 50$ samples per class) to measure data efficiency of each backbone.
4. **Scenario 4.4: Corruption Robustness**: Models trained under Scenario 4.1 are evaluated on inputs corrupted by Gaussian noise, motion blur, and brightness shift at varying severity levels.
5. **Scenario 4.5: Layer-Wise Probing**: Features extracted from early, middle, and final convolutional stages of each backbone are used to train shallow classifiers, revealing how feature quality evolves with network depth.

2 Experimental Setup

This section describes the implementation details common to all five scenarios, including data preprocessing, training configuration, and scenario-specific protocols.

2.1 Data Preprocessing and Augmentation

All images were loaded from the AID dataset in image-folder format and resized to 224×224 pixels as required by all three backbones. Pixel values were normalized using ImageNet statistics: mean $\mu = [0.485, 0.456, 0.406]$ and standard deviation $\sigma = [0.229, 0.224, 0.225]$ per channel.

Training-time augmentation used the following pipeline:

- Resize to 256×256, followed by random crop to 224×224.
- Random horizontal flip with probability 0.5.
- Color jitter: brightness ± 0.2, contrast ± 0.2, saturation ± 0.1.
- ToTensor and ImageNet normalization.

Validation-time transforms used a deterministic pipeline: resize to 256×256, center crop to 224×224, ToTensor, and ImageNet normalization. No random augmentation was applied at validation time to ensure consistent evaluation.

2.2 Training Configuration

A shared training configuration was applied across all scenarios unless overridden by scenario-specific settings. The key hyperparameters are summarized in Table 1.

The optimizer was AdamW, applied only over trainable parameters (frozen parameters are excluded). The learning rate followed a cosine annealing schedule decaying from the initial LR to zero over the full training duration. Label smoothing of $\epsilon = 0.05$ was applied to the cross-entropy loss to reduce overconfidence and improve generalization. Gradient norms were clipped to a maximum of 1.0 per step. All experiments used 4 DataLoader worker processes with pinned memory for efficient GPU data transfer.

Hyperparameter	Value
Batch size	64
Epochs (Scenarios 4.1, 4.2, 4.4)	30
Epochs (Scenario 4.3 few-shot)	20
Optimizer	AdamW
LR (linear probe / head)	1×10^{-3}
LR (fine-tuning backbone)	1×10^{-4}
Weight decay	1×10^{-4}
LR scheduler	CosineAnnealingLR
Loss function	Cross-entropy with label smoothing ($\epsilon = 0.05$)
Gradient clipping	Max norm = 1.0
Input resolution	224×224
Random seed	123

Table 1: Common hyperparameters used across all experiments.

2.3 Reproducibility

All experiments used a fixed global seed of 123, applied consistently to Python's `random` module, NumPy, and PyTorch (including CUDA). PyTorch's `cudnn.deterministic` flag was set to `True` and `cudnn.benchmark` was disabled to ensure fully deterministic behavior across runs. The dataset split (80/20 stratified) and all few-shot subsets were also drawn using the same seed, ensuring that results are exactly reproducible.

2.4 Scenario 4.1: Linear Probing

In the linear probing scenario, all backbone parameters were **frozen** immediately after loading the pretrained weights. Only the newly added 30-class fully connected classification head was trained. This setting isolates the quality of the pretrained feature representation without allowing any task-specific adaptation of the backbone.

Each model was trained for 30 epochs using AdamW with a learning rate of 1×10^{-3} and cosine annealing. Since only the linear head is trained, this scenario is computationally lightweight and serves as the baseline for all other comparisons. The best model checkpoint (by validation accuracy) was saved and used for downstream evaluation in Scenario 4.4.

2.5 Scenario 4.2: Selective Fine-Tuning

Selective fine-tuning partially unfreezes the backbone to allow task-specific adaptation. The constraint imposed was that **at most 20% of the backbone parameters** may be unfrozen. The specific layers unfrozen for each model were chosen to be the final convolutional stage:

- **ResNet50**: `layer4.2` (the last bottleneck block), 19.0% of backbone parameters.
- **EfficientNet-B0**: `blocks.6` (the final MBConv block group), 17.9% of backbone parameters.
- **ConvNeXt-Tiny**: `stages.3.blocks.2` (the last ConvNeXt block), 17.1% of backbone parameters.

A two-group optimizer was used: the classification head trained at LR $= 1 \times 10^{-3}$ and the unfrozen backbone layers at a lower LR $= 1 \times 10^{-4}$, both following cosine annealing over 30 epochs. Using a lower LR for the backbone avoids destroying the pretrained representations while still enabling adaptation to the aerial domain.

2.6 Scenario 4.3: Few-Shot Learning

The few-shot scenario evaluates how well each backbone's pretrained features generalize under severe data scarcity. For each model, the linear probe protocol from Scenario 4.1 was repeated (backbone fully frozen, only head trained) but with a **restricted fraction of the training set**. Three data regimes were tested:

- **Full data** (100%): 5,594 training samples, used as the full-data reference.
- **20%**: approximately 1,119 training samples.
- **5%**: approximately 280 training samples.

Subsets were drawn via stratified sampling with the fixed seed to ensure proportional class representation at every fraction. Each few-shot experiment was trained for 20 epochs with the same AdamW + CosineAnnealingLR configuration and a learning rate of 1×10^{-3}.

2.7 Scenario 4.4: Corruption Robustness

This scenario evaluates the robustness of the linearly probed models (from Scenario 4.1) under three types of input corruption applied **only at evaluation time**; no corruption-aware training was performed. The three corruption types are:

- **Gaussian Noise**: Zero-mean Gaussian noise with standard deviation $\sigma \in \{0.05, 0.10, 0.20\}$ is added to the normalized image tensor after standard preprocessing. Higher σ values represent stronger degradation.
- **Motion Blur**: A horizontal box-filter blur with kernel size 15 is applied to the raw PIL image before ToTensor and normalization. This simulates camera or platform motion during image capture.
- **Brightness Shift**: The image brightness is scaled by a factor of 1.5 (PIL-level, before normalization), simulating overexposure or varying illumination conditions.

For each corruption type and severity level, the best checkpoint from Scenario 4.1 was loaded and evaluated on the full validation set with the corresponding corrupted transform pipeline. Results are reported as top-1 accuracy under each corruption condition relative to the clean baseline.

2.8 Scenario 4.5: Layer-Wise Probing

Layer-wise probing investigates how the quality and discriminability of internal representations change across network depth. For each model, intermediate feature maps were extracted from three depth levels using forward hooks:

- **ResNet50**: `layer1` (early), `layer2` (middle), `layer4` (final).
- **EfficientNet-B0**: `blocks.0` (early), `blocks.3` (middle), `blocks.6` (final).
- **ConvNeXt-Tiny**: `stages.0` (early), `stages.1` (middle), `stages.3` (final).

Features were extracted from a fixed subset of **30 samples per class** (900 total) drawn from the validation set using the same seed across all models to ensure comparability. Spatial feature maps were reduced via global average pooling before being used as fixed representations. A linear classifier was then trained on top of each set of extracted features to measure linear separability at each depth level. In addition, PCA-based 2D projections were computed and visualized to qualitatively inspect how class structure evolves from early to final layers.

3 Linear Probing (Scenario 4.1)

3.1 Model Complexity

Before training, the computational complexity of each backbone was measured using the `ptflops` library on a single $224 \times 224 \times 3$ input. Table 2 reports the total parameter count, MACs (Multiply-Accumulate Operations), and FLOPs ($= 2 \times$ MACs) for each model. In the linear probing setting, only the classification head is trained; the number of trainable parameters is therefore very small relative to the total.

EfficientNet-B0 is by far the most parameter-efficient model, with only 4.05M total parameters and 0.39G MACs, roughly $10\times$ fewer MACs than ResNet50 and ConvNeXt-Tiny. ConvNeXt-Tiny has the largest total parameter count (27.84M) but the fewest trainable parameters in the linear probe setting (24,606), owing to its compact classifier head design. ResNet50 falls in between across all metrics.

Model	Total Params	Trainable Params	MACs	FLOPs
ResNet50	23.57M	61,470	4.13G	8.26G
EfficientNet-B0	4.05M	38,430	0.39G	0.78G
ConvNeXt-Tiny	27.84M	24,606	4.48G	8.96G

Table 2: Model complexity metrics at 224×224 input resolution. Trainable parameters correspond to the linear probe setting (frozen backbone, head only).

3.2 Protocol

In the linear probing scenario, all backbone parameters were frozen after loading ImageNet-pretrained weights from `timm`. Only the newly added 30-class fully connected classification head was trained. This setting directly measures the quality of the pretrained representation without any domain adaptation of the backbone features.

Training proceeded for 30 epochs using the AdamW optimizer with a learning rate of 1×10^{-3} and cosine annealing. The loss function was cross-entropy with label smoothing ($\epsilon = 0.05$) and gradients were clipped to a maximum norm of 1.0. The best model checkpoint (by validation accuracy) was retained for evaluation and for downstream use in Scenario 4.4.

3.3 Training and Validation Accuracy Curves

Figures 1–3 show the training and validation accuracy curves over 30 epochs for each model.

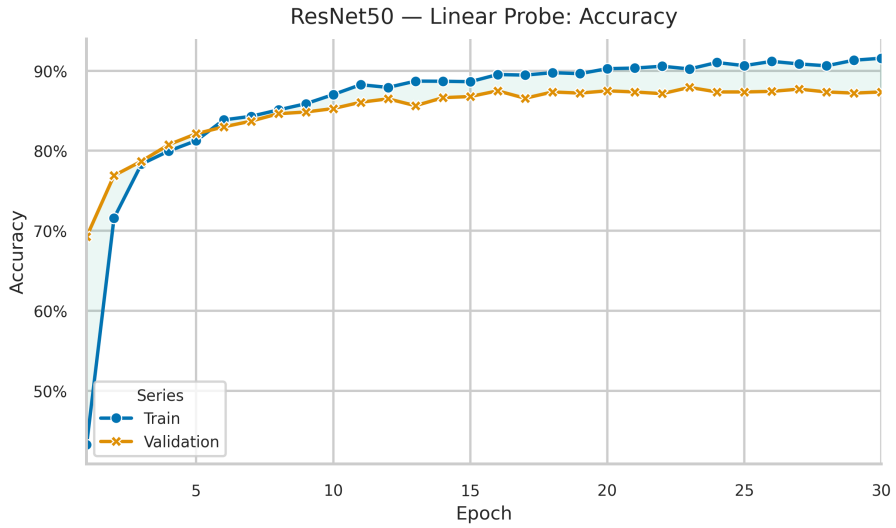

Figure 1: ResNet50, Linear Probe: Training and validation accuracy curves.

All three models converge within the 30-epoch budget. ConvNeXt-Tiny reaches the highest best validation accuracy of **94.85%** at epoch 16, followed by ResNet50 at **87.92%** (epoch 23) and EfficientNet-B0 at **86.20%** (epoch 27). A notable observation is the large gap between training and validation accuracy in all models: training accuracy reaches 93.4–99.7% while validation accuracy plateaus in the 86–94% range. This gap is expected in a linear probe: the head quickly saturates on the training distribution, but the frozen backbone cannot be adapted to reduce the generalization gap. The gap is smallest for ConvNeXt-Tiny, suggesting its representations are the most transferable to the aerial domain.

Table 3 summarizes the final performance of each model.

3.4 Confusion Matrix

Figures 4–6 show the per-class confusion matrices on the validation set for each model.

Across all three models, the diagonal is dominant, indicating strong overall classification. However, off-diagonal confusions concentrate in a consistent set of visually similar class pairs. Common failure

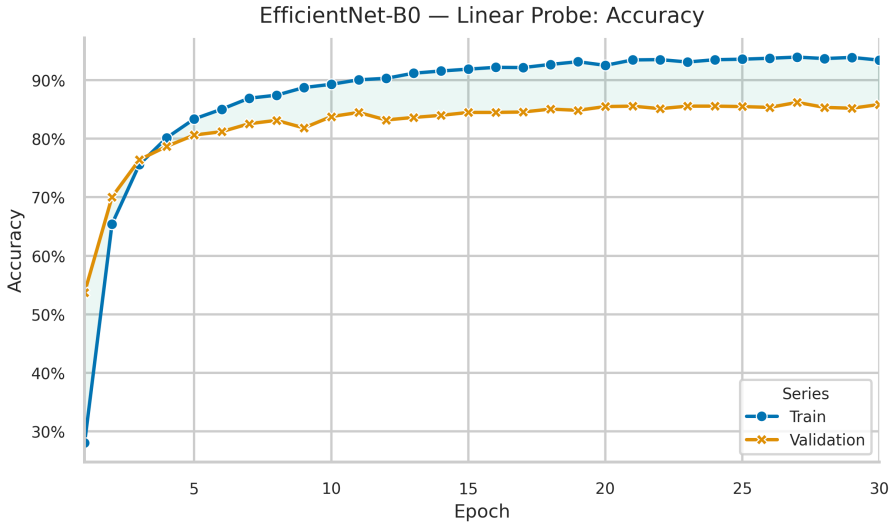

Figure 2: EfficientNet-B0, Linear Probe: Training and validation accuracy curves.

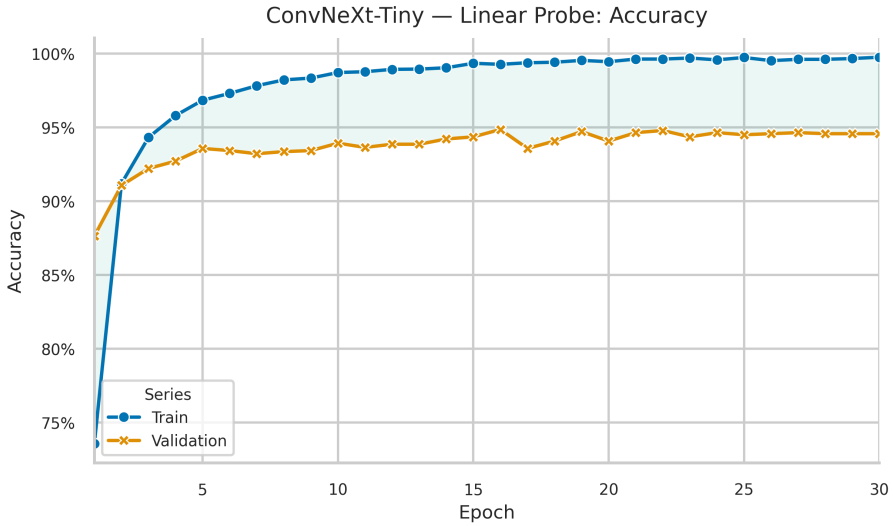

Figure 3: ConvNeXt-Tiny, Linear Probe: Training and validation accuracy curves.

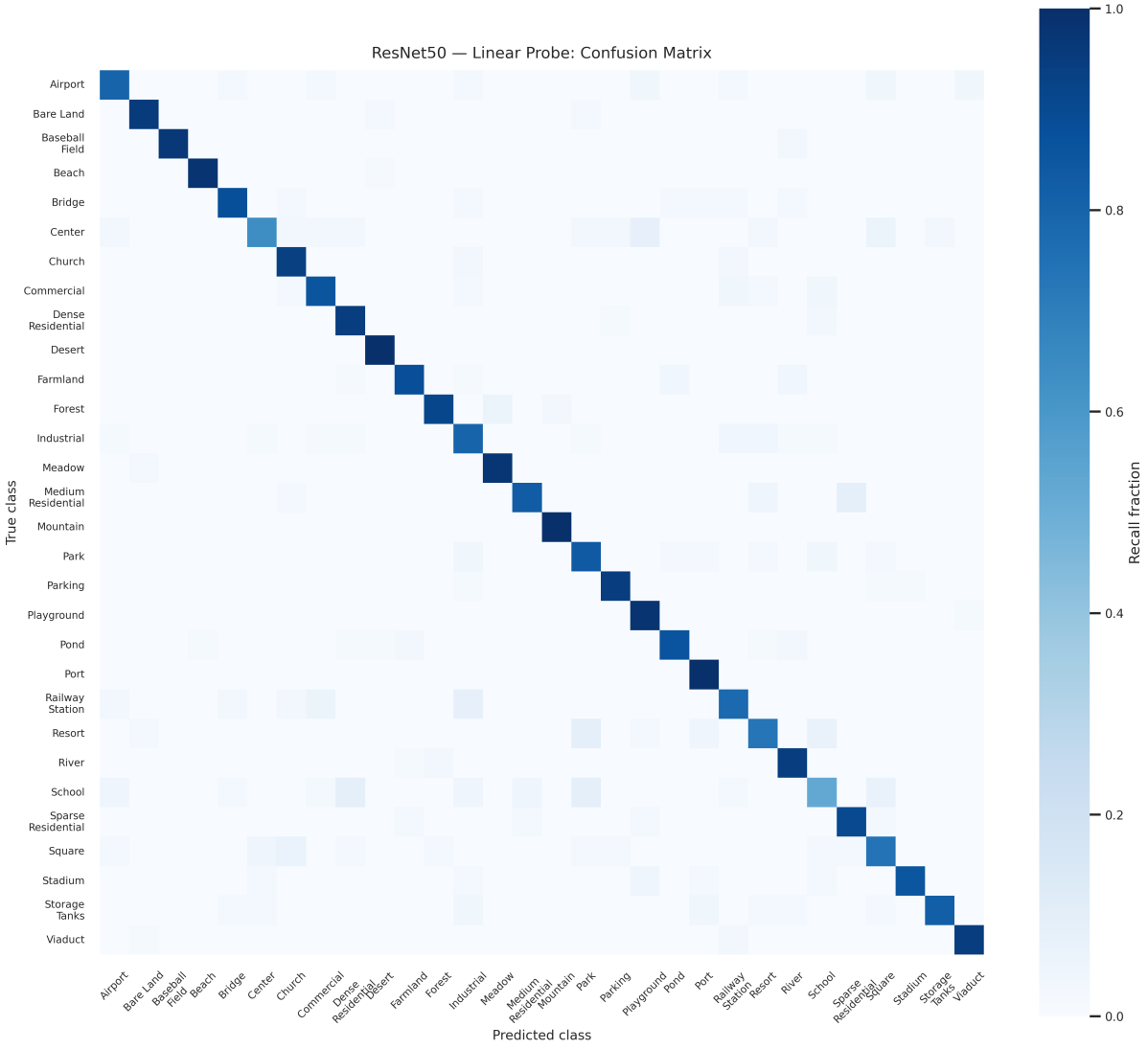

Figure 4: ResNet50, Linear Probe: Confusion matrix on the validation set.

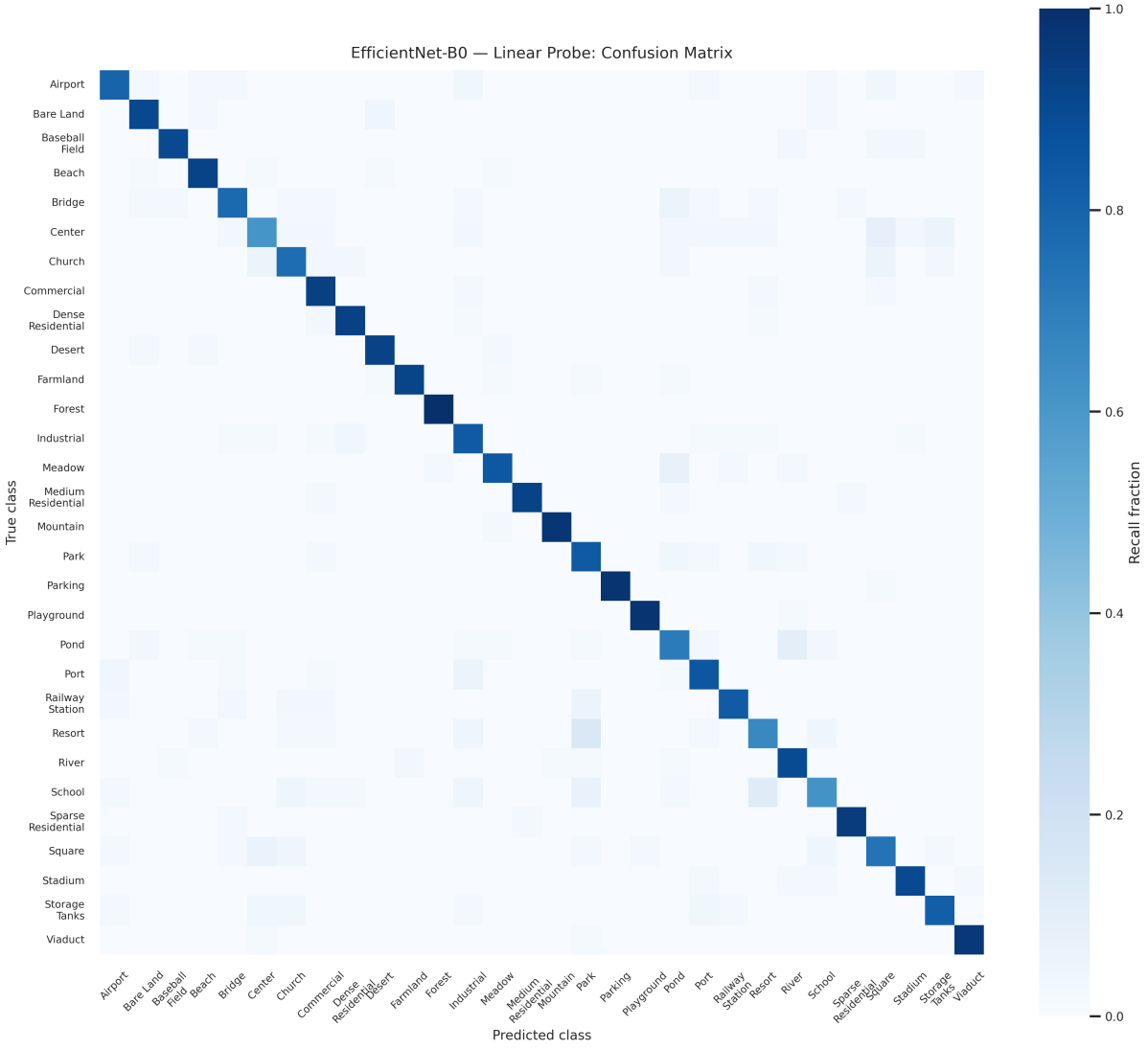

Figure 5: EfficientNet-B0, Linear Probe: Confusion matrix on the validation set.

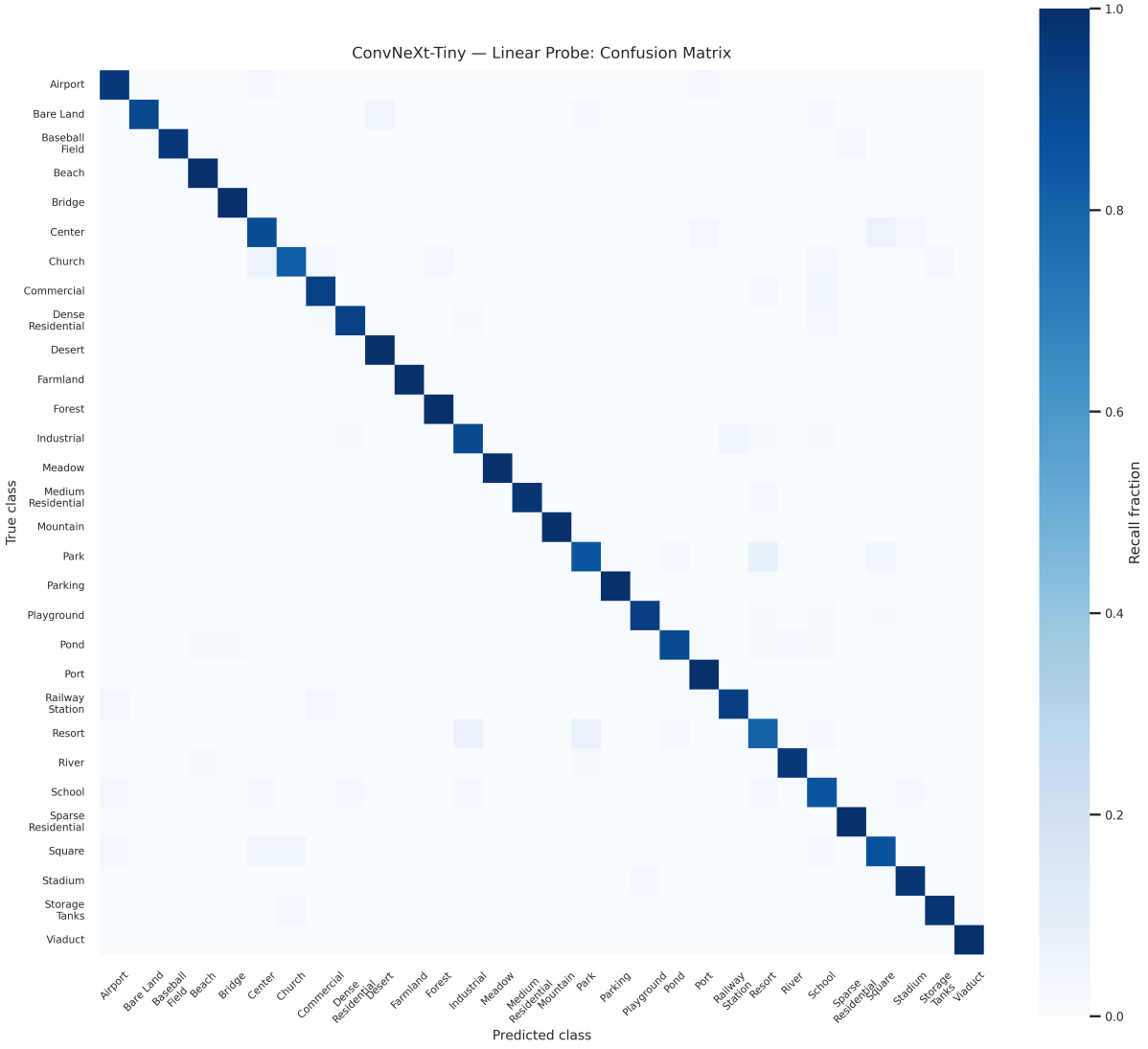

Figure 6: ConvNeXt-Tiny, Linear Probe: Confusion matrix on the validation set.

Model	Best Val Acc (%)	Best Epoch	Final Train Acc (%)	Train–Val Gap (%)
ResNet50	87.92	23	91.56	4.07
EfficientNet-B0	86.20	27	93.42	7.57
ConvNeXt-Tiny	**94.85**	16	99.75	5.69

Table 3: Linear probe performance summary across all three models.

cases include:

- **Meadow** ↔ **Farmland**: Both classes share green, textured ground cover with no strong structural landmarks, making them difficult to separate from frozen ImageNet features alone.
- **Beach** ↔ **Desert**: Both exhibit large homogeneous sandy/light regions; without color calibration or structural context, frozen features from ImageNet may conflate them.
- **Park** ↔ **Forest**: Partial tree canopy coverage and similar green texture cause confusion in weaker models (ResNet50, EfficientNet-B0).
- **Commercial** ↔ **Industrial**: Dense building layouts with similar roof textures and spatial arrangements are frequently confused by ResNet50 and EfficientNet-B0.

ConvNeXt-Tiny shows the cleanest diagonal with the fewest off-diagonal confusions, consistent with its higher overall accuracy. ResNet50 and EfficientNet-B0 exhibit more diffuse confusion patterns, particularly for the texture-dominated scene pairs above.

3.5 Feature Embedding Visualization (t-SNE)

Figures 7–9 show t-SNE projections of the final-layer backbone features extracted from the full validation set. Features were extracted using a forward hook on the last convolutional stage of each model and reduced to 2D using t-SNE with the same seed (123) for reproducibility.

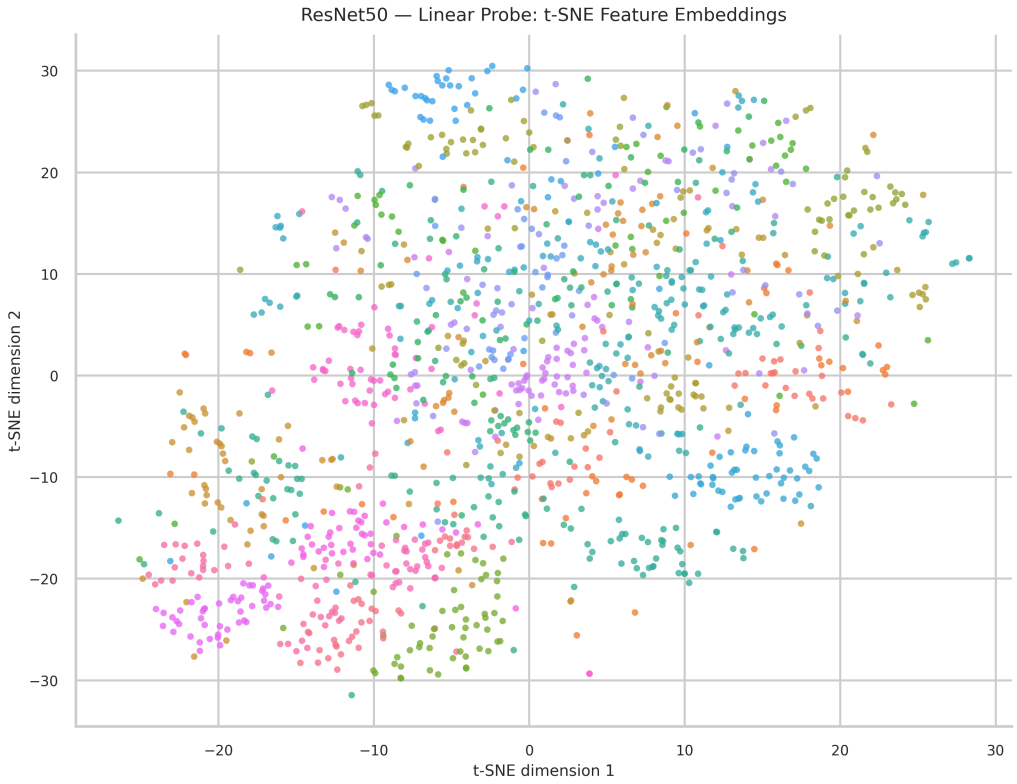

Figure 7: ResNet50, Linear Probe: t-SNE visualization of final-layer features on the validation set.

Figure 8: EfficientNet-B0, Linear Probe: t-SNE visualization of final-layer features on the validation set.

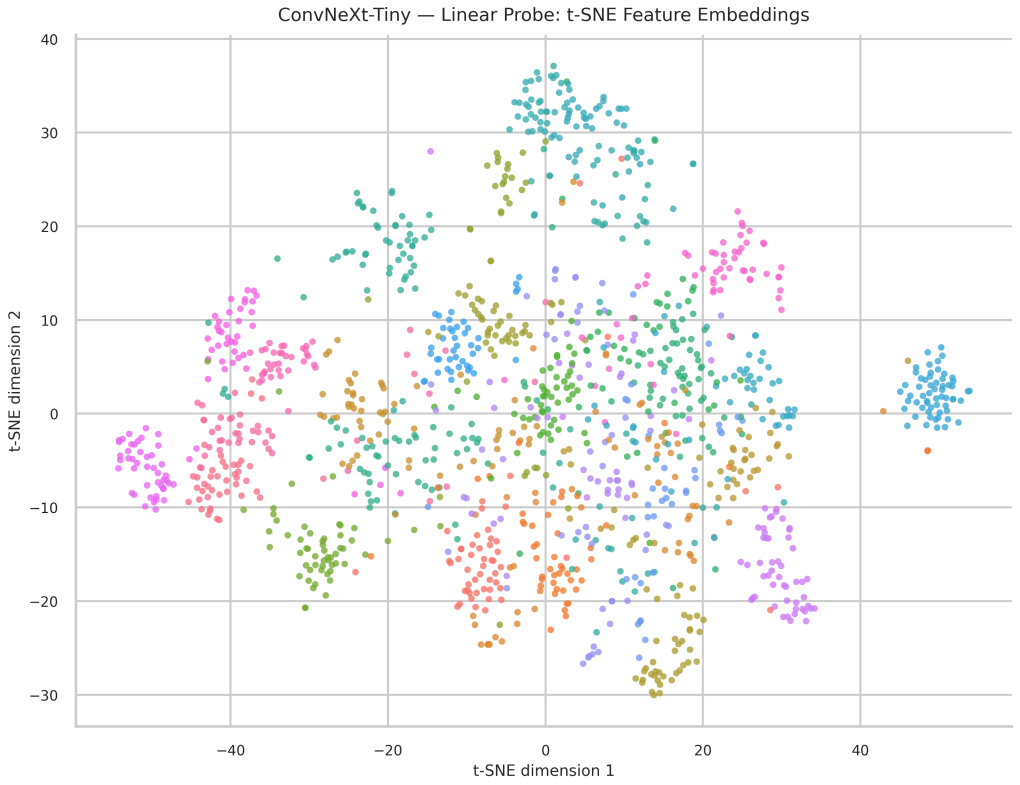

Figure 9: ConvNeXt-Tiny, Linear Probe: t-SNE visualization of final-layer features on the validation set.

3.6 Analysis: Feature Separability and Transfer Quality

- **ConvNeXt-Tiny provides the strongest frozen transfer.** Its best validation accuracy reaches 94.85%, which is substantially higher than ResNet50 (87.92%) and EfficientNet-B0 (86.20%). This indicates that its ImageNet-pretrained representation already aligns well with aerial-scene semantics before any backbone adaptation.
- **The t-SNE plots support the accuracy ranking.** ConvNeXt-Tiny shows the cleanest feature grouping and the least overlap among major scene categories, while ResNet50 shows moderate overlap and EfficientNet-B0 shows the most mixing. The visual evidence is therefore consistent with the quantitative ordering.
- **ResNet50 remains a solid baseline, but its features are less separable for visually similar classes.** The confusion patterns and t-SNE overlap suggest that texture-heavy or land-use-related categories are not as cleanly separated as they are in ConvNeXt-Tiny.
- **EfficientNet-B0's weakness is not only lower capacity.** It has the lowest validation accuracy and the largest train–val gap, which suggests that its frozen representation is less transferable to AID rather than simply under-parameterized.
- **The remaining errors are semantically plausible.** Overlaps among classes such as park-like, residential, water, and urban-center scenes are expected in overhead imagery, so even the best frozen backbone does not completely remove class ambiguity.

4 Fine-Tuning Strategies (Scenario 4.2)

4.1 Protocol

Scenario 4.2 extends the linear probe by progressively unfreezing portions of the backbone, allowing task-specific adaptation of the pretrained features. Four strategies were evaluated for each model:

- **Linear Probe**: Backbone fully frozen; only the 30-class head is trained. Serves as the within-scenario baseline (identical to Scenario 4.1 but re-run with the two-group optimizer for fair comparison).
- **Last Block**: The final convolutional block of the backbone is unfrozen in addition to the head.
- **Selective**: The final stage is unfrozen subject to a hard constraint of at most 20% of backbone parameters, chosen by gradient-magnitude calibration over 3 batches.
- **Full Fine-Tuning**: All backbone parameters are unfrozen alongside the head.

A two-group AdamW optimizer was used for all strategies: the classification head trained at LR $= 1 \times 10^{-3}$ and any unfrozen backbone layers at a lower LR $= 1 \times 10^{-4}$, both following cosine annealing over 30 epochs. Using a reduced backbone LR prevents catastrophic forgetting of the pretrained representations while still enabling aerial-domain adaptation.

Table 4 lists the exact layers unfrozen and the resulting fraction of backbone parameters made trainable for each model under the selective and last-block strategies.

Model	Strategy	Unfrozen Layer	Backbone % Unfrozen
ResNet50	Last Block	`layer4`	63.7%
ResNet50	Selective	`layer4.2`	19.9%
EfficientNet-B0	Last Block	`blocks.6`	17.9%
EfficientNet-B0	Selective	`blocks.6`	20.0%
ConvNeXt-Tiny	Last Block	`stages.3`	55.6%
ConvNeXt-Tiny	Selective	`stages.3.blocks.2`	19.4%

Table 4: Layers unfrozen and percentage of backbone parameters made trainable for last-block and selective strategies.

4.2 Gradient-Budgeted Selective Unfreezing

The selective strategy uses a gradient-based scoring procedure to decide which backbone groups to unfreeze, subject to a hard parameter budget. Algorithm 10 describes the full procedure.

Algorithm 1: Gradient-Budgeted Selective Unfreezing

Input: pretrained model $\mathcal{M}$, calibration loader $\mathcal{D}_{\text{cal}}$, budget fraction α, calibration batches B
Output: selected layer groups $\mathcal{S}$

Notation: $\mathcal{C} = \{g_i\}$ is the ordered candidate set of layer groups, $|g_i|$ is the parameter count of group g_i, r_i is its depth rank, and ϕ_i is its final selection score.

Step 1: Budget computation

$P_{\text{backbone}} \leftarrow$ total backbone parameters of $\mathcal{M}$
$P_{\text{budget}} \leftarrow \lfloor \alpha P_{\text{backbone}} \rfloor$

Step 2: Candidate construction

Build ordered candidate groups $\mathcal{C}$ from deepest to shallowest
Temporarily enable gradients for all $g_i \in \mathcal{C}$

Step 3: Calibration scoring

For B mini-batches from $\mathcal{D}_{\text{cal}}$, compute loss and backpropagate
For each group g_i, measure the mean gradient norm and average it across batches to obtain $\bar{s}_i$

Step 4: Composite score

Freeze the backbone again
For each group g_i, compute:

$$\tilde{s}_i = \frac{\bar{s}_i}{\max_j \bar{s}_j}, \qquad \tilde{d}_i = \frac{\bar{s}_i/|g_i|}{\max_j(\bar{s}_j/|g_j|)}, \qquad \delta_i = 1 - \frac{r_i}{|\mathcal{C}| - 1}$$

$$\phi_i = 0.65\,\tilde{s}_i + 0.25\,\tilde{d}_i + 0.10\,\delta_i$$

Step 5: Greedy budgeted selection

Sort groups by decreasing ϕ_i
Add groups one by one while the total selected parameters remain within P_{budget}
If no group fits, choose the smallest candidate group

Step 6: Apply selection

Unfreeze all groups in $\mathcal{S}$ and return $\mathcal{S}$

Figure 10: Short description of the gradient-budgeted selective unfreezing method used in Scenario 4.2.

The composite score ϕ_i balances three signals. The raw gradient score $\tilde{s}_i$ captures the overall adaptation pressure on the group. The gradient density $\tilde{d}_i$ favors compact groups that carry high gradient signal per parameter, preventing the budget from being dominated by large but low-signal blocks. The depth bonus δ_i gives a small preference to deeper layers, which are known to encode more task-specific features. The weights $0.65 : 0.25 : 0.10$ were chosen to prioritise gradient magnitude while penalising parameter-inefficient choices. The greedy knapsack in Step 5 then selects the highest-scoring groups that fit within the budget $P_{budget} = \alpha \cdot P_{backbone}$, with $\alpha = 0.20$ in all experiments.

4.3 Results

Table 5 reports the best validation accuracy, trainable parameter count, and accuracy per million tuned parameters for each model-strategy combination.

4.4 Accuracy Curves Across Strategies

Figures 11–13 show the validation accuracy curves for all four strategies overlaid on the same plot for each model.

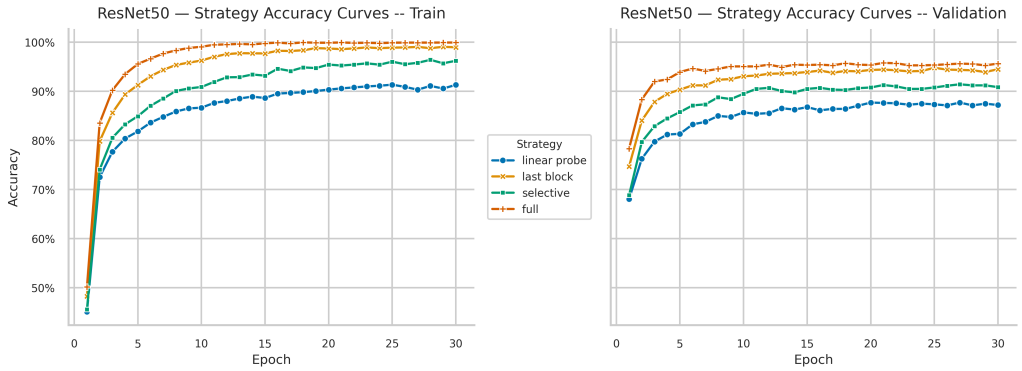

Figure 11: ResNet50, Scenario 4.2: Validation accuracy curves for all four fine-tuning strategies.

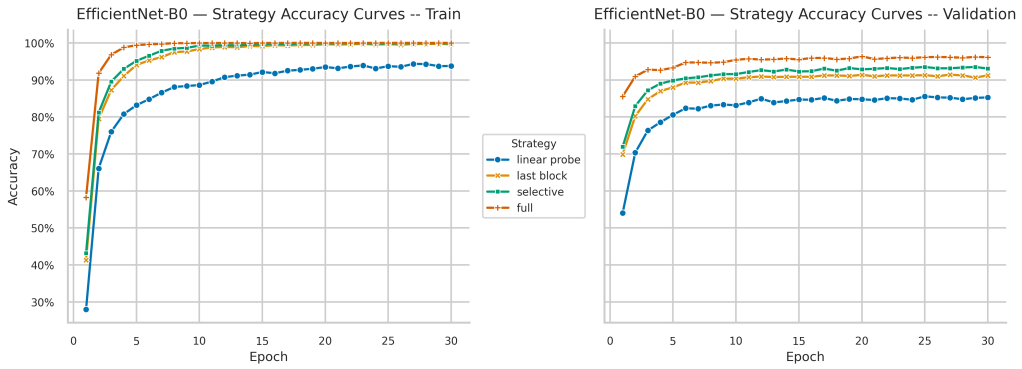

Figure 12: EfficientNet-B0, Scenario 4.2: Validation accuracy curves for all four fine-tuning strategies.

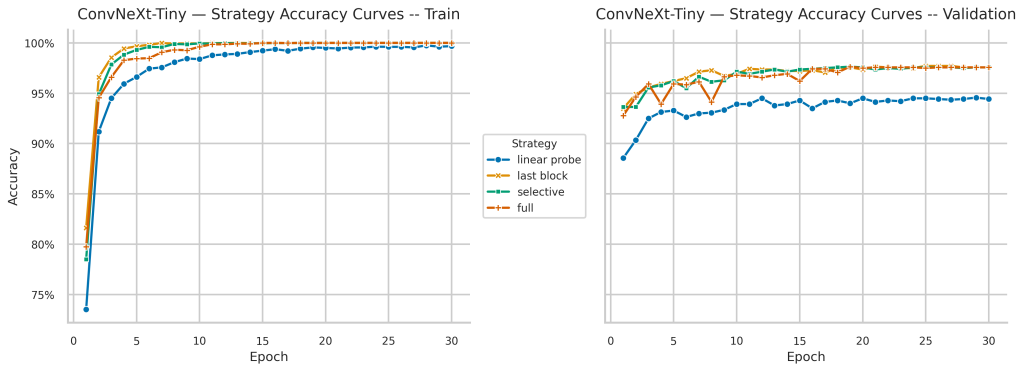

Figure 13: ConvNeXt-Tiny, Scenario 4.2: Validation accuracy curves for all four fine-tuning strategies.

Model	Strategy	Trainable Params	Best Val Acc (%)	Train Acc (%)	Acc/M Params
ResNet50	Linear Probe	61,470	87.71	91.31	1426.8
ResNet50	Last Block	15,026,206	94.78	98.96	6.3
ResNet50	Selective	4,749,406	91.42	96.21	19.2
ResNet50	Full	23,569,502	95.78	99.93	4.1
EffNet-B0	Linear Probe	38,430	85.56	93.76	2226.4
EffNet-B0	Last Block	755,662	91.42	99.64	121.0
EffNet-B0	Selective	839,712	93.50	99.82	111.3
EffNet-B0	Full	4,045,978	96.35	100.00	23.8
ConvNeXt-T	Linear Probe	24,606	94.57	99.68	3843.3
ConvNeXt-T	Last Block	15,495,198	97.71	100.00	6.3
ConvNeXt-T	Selective	5,424,126	**97.64**	100.00	18.0
ConvNeXt-T	Full	27,843,198	97.64	100.00	3.5

Table 5: Fine-tuning strategy comparison. Acc/M Params = best validation accuracy (%) per million trainable parameters.

4.5 Accuracy vs. Unfrozen Parameters

Figures 14–16 plot best validation accuracy against the fraction of backbone parameters unfrozen, illustrating the marginal benefit of unlocking additional parameters.

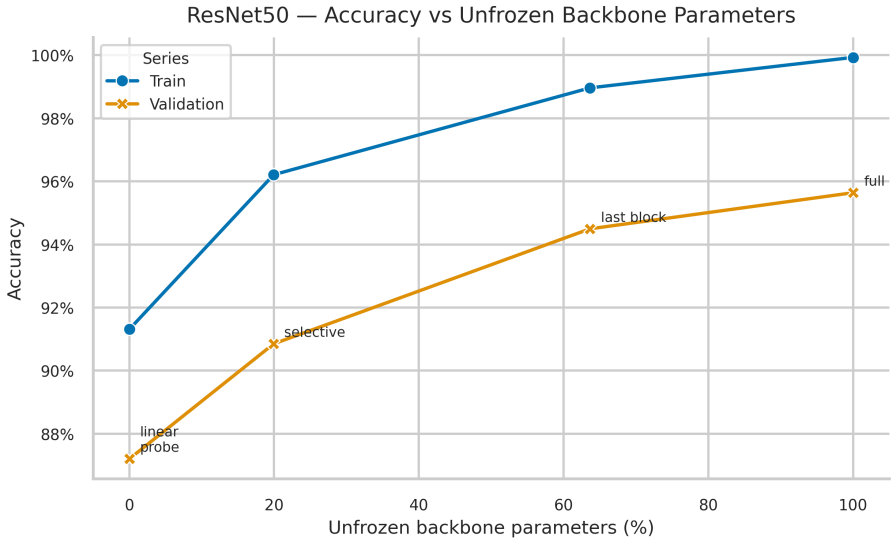

Figure 14: ResNet50, Scenario 4.2: Best validation accuracy vs. percentage of backbone parameters unfrozen.

4.6 Convergence Loss Comparison

Figures 17–19 show the training loss curves for all strategies on each model, illustrating how quickly each strategy converges and whether full fine-tuning leads to faster or more stable convergence.

4.7 Analysis

- **Unfreezing more of the backbone consistently improves accuracy, but only up to a point.** All three models outperform linear probing once some backbone adaptation is allowed, confirming that frozen ImageNet features leave useful headroom on AID.
- **ConvNeXt-Tiny shows the clearest saturation effect.** Its selective strategy already reaches 97.64% while tuning only 19.4% of the backbone, which matches full fine-tuning. This means full adaptation is unnecessary for ConvNeXt-Tiny on this dataset.

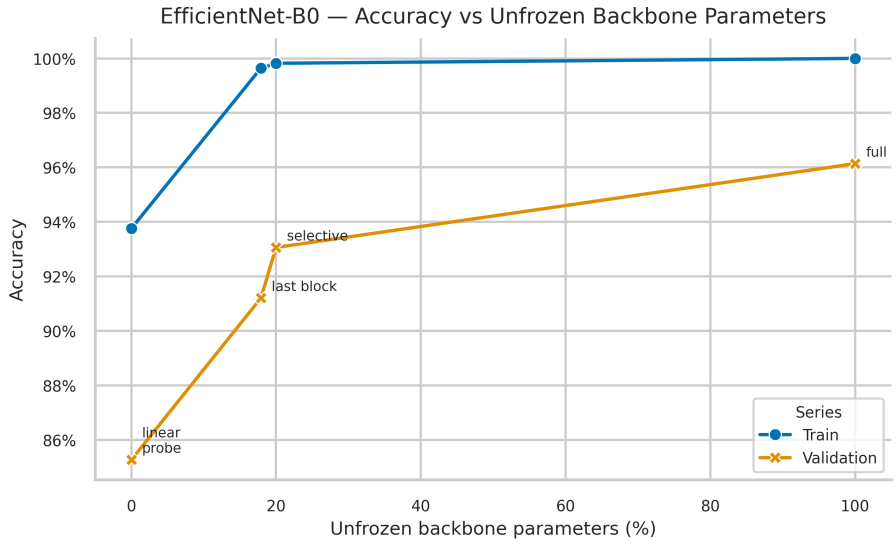

Figure 15: EfficientNet-B0, Scenario 4.2: Best validation accuracy vs. percentage of backbone parameters unfrozen.

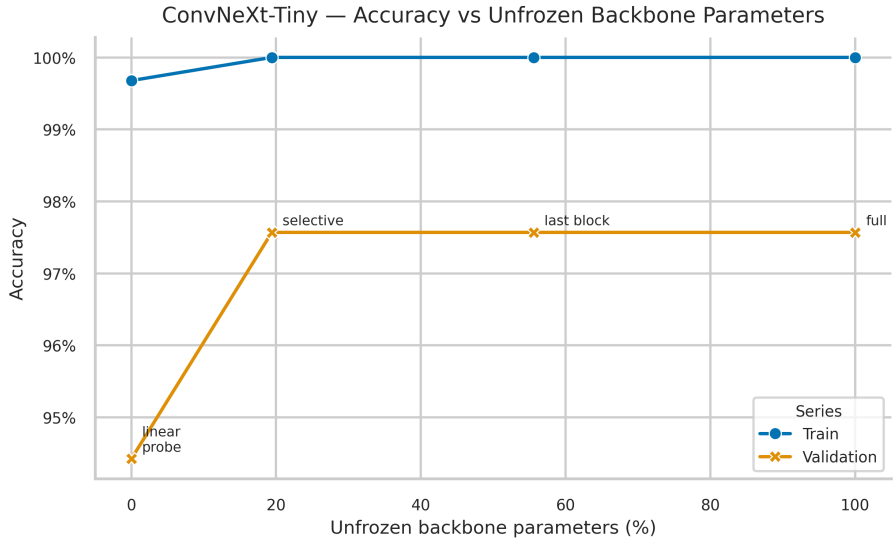

Figure 16: ConvNeXt-Tiny, Scenario 4.2: Best validation accuracy vs. percentage of backbone parameters unfrozen.

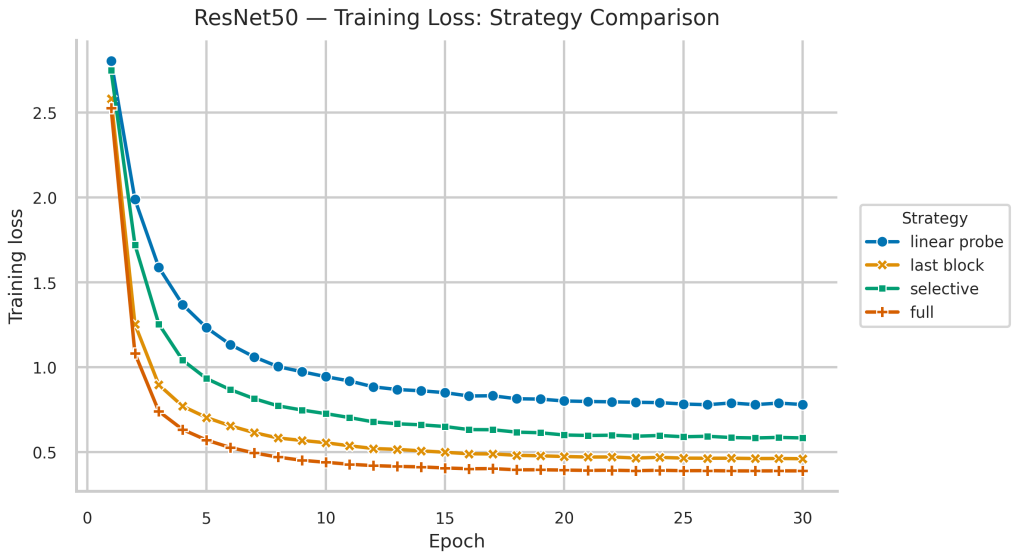

Figure 17: ResNet50, Scenario 4.2: Training loss curves for all four strategies.

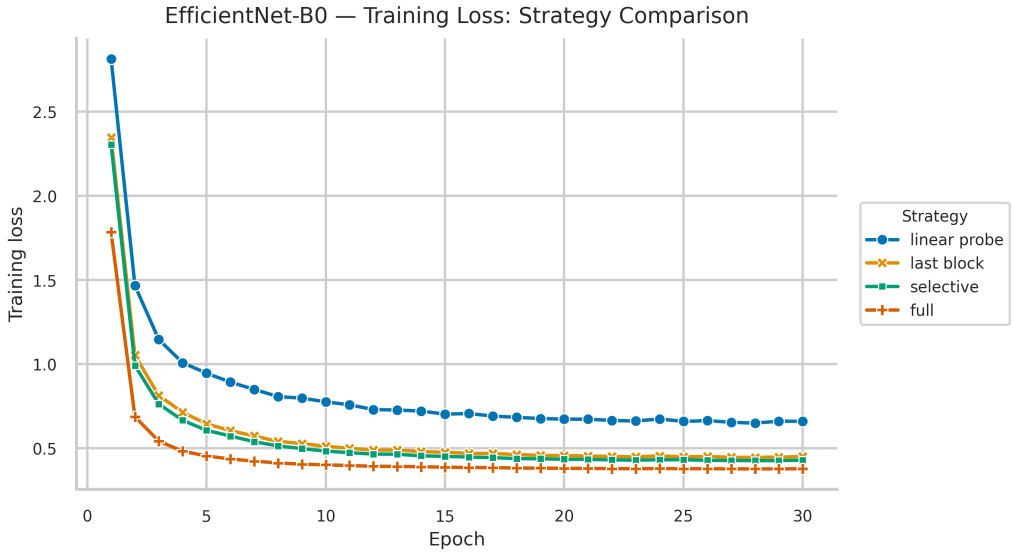

Figure 18: EfficientNet-B0, Scenario 4.2: Training loss curves for all four strategies.

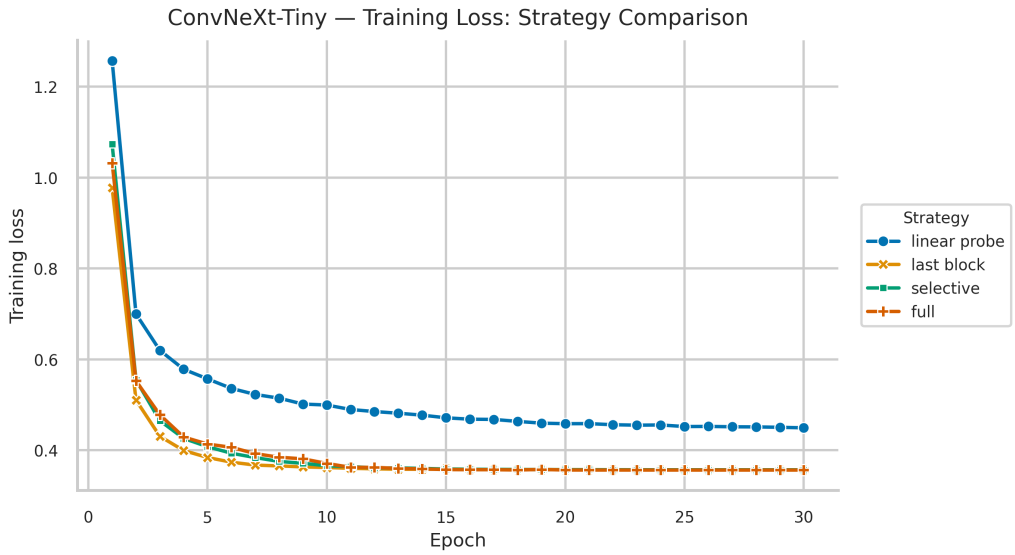

Figure 19: ConvNeXt-Tiny, Scenario 4.2: Training loss curves for all four strategies.

- **Selective tuning gives the best parameter-efficiency trade-off.** The gain is strongest for ConvNeXt-Tiny and still meaningful for EfficientNet-B0, showing that carefully targeted adaptation can recover most of the benefit of full fine-tuning at a much lower trainable-parameter cost.
- **ResNet50 and EfficientNet-B0 still benefit from broader adaptation.** Unlike ConvNeXt-Tiny, they continue to improve when more of the backbone is unfrozen, which suggests their pretrained features are farther from AID-specific saturation.
- **Generalization quality still differs across models.** ConvNeXt-Tiny maintains the smallest train–val gap under strong performance, whereas EfficientNet-B0 continues to show a relatively larger gap, indicating weaker adaptation stability even after tuning.

5 Few-Shot Learning (Scenario 4.3)

5.1 Protocol

Scenario 4.3 evaluates how well each backbone's pretrained features generalize when labeled training data is severely limited. The linear probe strategy from Scenario 4.1 is used throughout: the backbone remains fully frozen and only the 30-class classification head is trained. This isolates data efficiency as the sole variable.

Three training data regimes were evaluated by subsampling the 5,594-image training split:

- **100%**: 5,594 samples (full training set, 30-epoch reference).
- **20%**: 1,118 samples, approximately 37 images per class.
- **5%**: 279 samples, approximately 9 images per class.

Subsets were drawn by stratified sampling with seed 123 to preserve class balance at every fraction. The full-data (100%) regime was trained for 30 epochs; the low-data regimes (20%, 5%) were trained for 20 epochs with the same AdamW + CosineAnnealingLR setup and learning rate 1×10^{-3}. The validation set was always the fixed 1,399-image split, ensuring consistent evaluation across all regimes.

5.2 Results

Table 6 reports the best validation accuracy, final training accuracy, and train-validation gap for each model at every data fraction. The relative performance drop Δ measures the degradation from 100% to 5% data as a fraction of the 100% accuracy.

The relative performance drops from 100% to 5% data are: ResNet50 $\Delta = 33.4\%$, EfficientNet-B0 $\Delta = 60.5\%$, and ConvNeXt-Tiny $\Delta = 18.5\%$. ConvNeXt-Tiny is by far the most data-efficient backbone.

Model	Fraction	N Samples	Best Val Acc (%)	Train Acc (%)	Gap (%)
ResNet50	100%	5,594	87.92	91.56	4.21
ResNet50	20%	1,118	77.41	87.75	10.91
ResNet50	5%	279	58.54	86.74	28.70
EfficientNet-B0	100%	5,594	86.20	93.42	7.57
EfficientNet-B0	20%	1,118	67.62	87.57	19.95
EfficientNet-B0	5%	279	34.10	61.65	27.55
ConvNeXt-Tiny	100%	5,594	**94.85**	99.75	5.18
ConvNeXt-Tiny	20%	1,118	**87.92**	100.00	12.51
ConvNeXt-Tiny	5%	279	**77.34**	99.64	22.73

Table 6: Few-shot linear probe results. Gap = Final Train Acc − Final Val Acc. ConvNeXt-Tiny dominates at every data fraction.

5.3 Accuracy vs. Data Fraction

Figure 20 shows best validation accuracy as a function of training data fraction for all three models on a single plot.

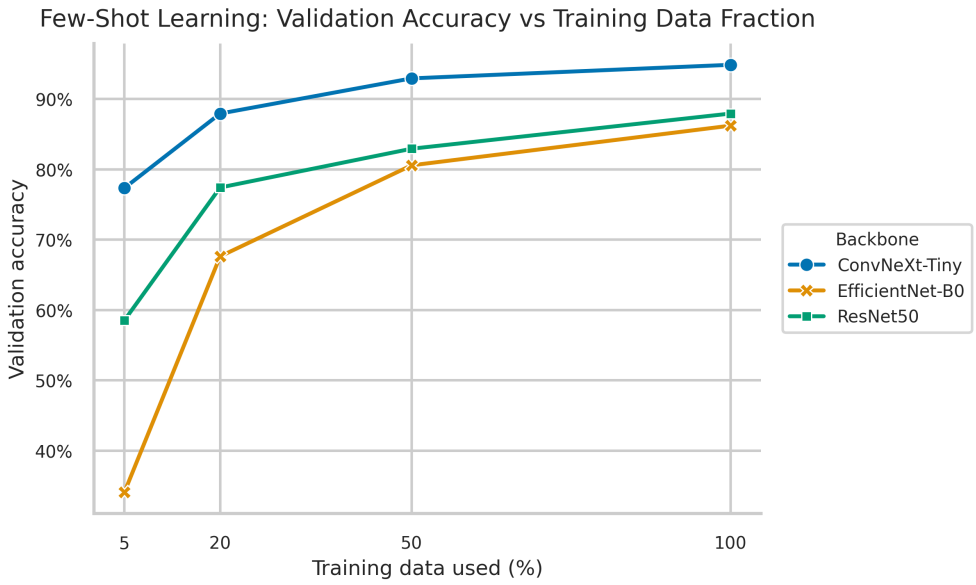

Figure 20: Few-shot linear probe: best validation accuracy vs. training data fraction for all three backbones.

Figure 21 shows the relative accuracy drop Δ from 100% to 5% data for each backbone, summarising data efficiency in a single metric.

5.4 Analysis

- **ConvNeXt-Tiny is the most data-efficient backbone.** Even with only 5% of the training data, it reaches 77.34% validation accuracy, clearly outperforming ResNet50 (58.54%) and EfficientNet-B0 (34.10%).
- **Relative degradation matters as much as absolute accuracy.** ConvNeXt-Tiny suffers the smallest drop from the full-data regime, ResNet50 degrades moderately, and EfficientNet-B0 collapses sharply. This shows that ConvNeXt-Tiny preserves useful class structure even when supervision is scarce.
- **Model size alone does not explain the ranking.** ConvNeXt-Tiny is also the largest model here, yet it is the strongest under low data. The more convincing explanation is that its pretrained

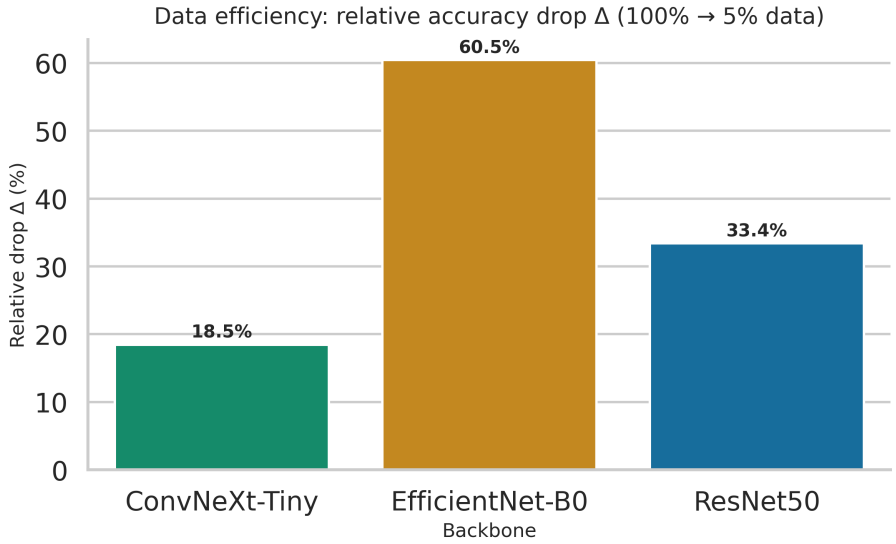

Figure 21: Data efficiency: relative accuracy drop Δ from 100% to 5% training data per backbone. Lower is better. ConvNeXt-Tiny ($\Delta = 18.5\%$) is substantially more robust to data reduction than ResNet50 ($\Delta = 33.4\%$) and EfficientNet-B0 ($\Delta = 60.5\%$).

representation is better aligned with the aerial domain, so the linear head needs less target-specific evidence.

- **Overfitting becomes severe as supervision decreases.** The train–val gap widens substantially for all models in the 20% and especially 5% regimes, confirming that low-data transfer is limited not just by optimization but by weak statistical support.
- **EfficientNet-B0 is the least reliable choice when labels are scarce.** Its sharp accuracy collapse indicates that its frozen features are comparatively brittle in the few-shot regime.

6 Corruption Robustness (Scenario 4.4)

6.1 Protocol

This scenario evaluates how well the linearly probed models from Scenario 4.1 withstand controlled distribution shifts applied only at evaluation time. No corruption-aware training was performed; models see corrupted images only during inference. Three corruption types were applied to the full 1,399-image validation set:

- **Gaussian noise** ($\sigma \in \{0.05, 0.1, 0.2\}$): zero-mean Gaussian noise added to the normalized image tensor after standard preprocessing. Higher σ represents progressively stronger pixel-level degradation.
- **Motion blur** (kernel size $k = 15$): horizontal box-filter blur applied at the PIL level before normalization, simulating camera or platform motion during aerial capture.
- **Brightness shift** (factor $f = 1.5$): image brightness scaled by $1.5\times$ at the PIL level before normalization, simulating overexposure or variable illumination.

Two metrics are reported per condition, as defined in the assignment specification:

$$\text{Corruption Error} = 1 - \text{Acc}_{\text{corrupted}}$$

$$\text{Relative Robustness} = \frac{\text{Acc}_{\text{corrupted}}}{\text{Acc}_{\text{clean}}}$$

Relative Robustness close to 1.0 indicates the model retains most of its clean accuracy under corruption.

6.2 Results

Table 7 reports the corrupted accuracy, corruption error, and relative robustness for all three models across every corruption condition.

Model	Corruption	Clean Acc	Corr. Acc	Corr. Error	Rel. Robust.
ResNet50	Gaussian σ=0.05	87.92	78.27	0.217	0.890
ResNet50	Gaussian σ=0.10	87.92	71.69	0.283	0.815
ResNet50	Gaussian σ=0.20	87.92	28.45	0.716	0.324
ResNet50	Motion Blur	87.92	36.24	0.638	0.412
ResNet50	Brightness f=1.5	87.92	83.85	0.162	0.954
EffNet-B0	Gaussian σ=0.05	86.20	85.28	0.147	0.989
EffNet-B0	Gaussian σ=0.10	86.20	78.98	0.210	0.916
EffNet-B0	Gaussian σ=0.20	86.20	28.31	0.717	0.328
EffNet-B0	Motion Blur	86.20	25.73	0.743	0.299
EffNet-B0	Brightness f=1.5	86.20	80.84	0.192	0.938
ConvNeXt-T	Gaussian σ=0.05	94.85	93.42	0.066	0.985
ConvNeXt-T	Gaussian σ=0.10	94.85	91.92	0.081	0.969
ConvNeXt-T	Gaussian σ=0.20	94.85	84.92	0.151	0.895
ConvNeXt-T	Motion Blur	94.85	45.75	0.543	0.482
ConvNeXt-T	Brightness f=1.5	94.85	93.71	0.063	0.988

Table 7: Corruption robustness results. Corr. Error = $1 - \text{Acc}_{\text{corrupted}}$. Rel. Robust. = $\text{Acc}_{\text{corrupted}}/\text{Acc}_{\text{clean}}$.

6.3 Visualizations

Figure 22 shows corrupted accuracy grouped by corruption type across all three models, and Figure 23 shows the corresponding relative robustness. Figure 24 provides an aggregate summary heatmap across all conditions.

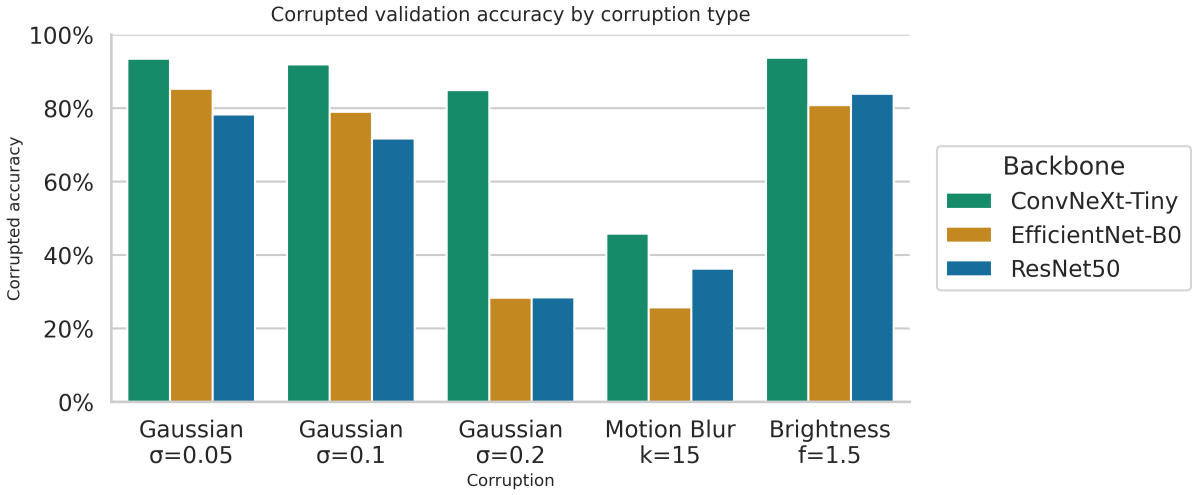

Figure 22: Corrupted validation accuracy by corruption type and severity for all three models.

6.4 Analysis

- **ConvNeXt-Tiny is the most robust model overall.** It achieves the strongest aggregate relative robustness and remains especially stable under Gaussian noise and brightness shift.
- **Motion blur is the universal failure mode.** All three models suffer their largest drops under blur, which suggests that directional smearing removes edge and texture cues that the pretrained backbones still rely on heavily.

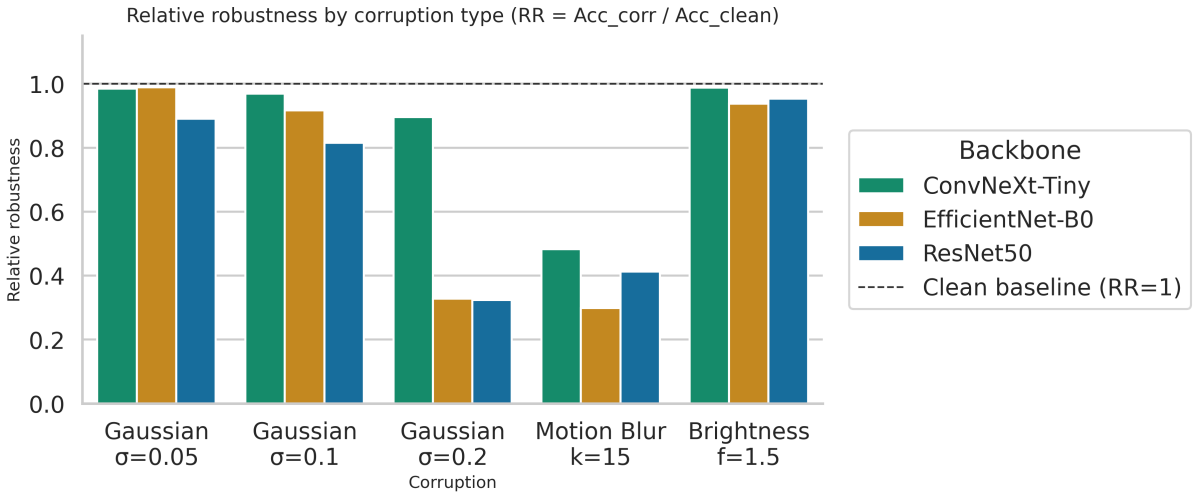

Figure 23: Relative robustness ($\text{Acc}_{\text{corrupted}}/\text{Acc}_{\text{clean}}$) by corruption type. Values closer to 1.0 indicate better robustness.

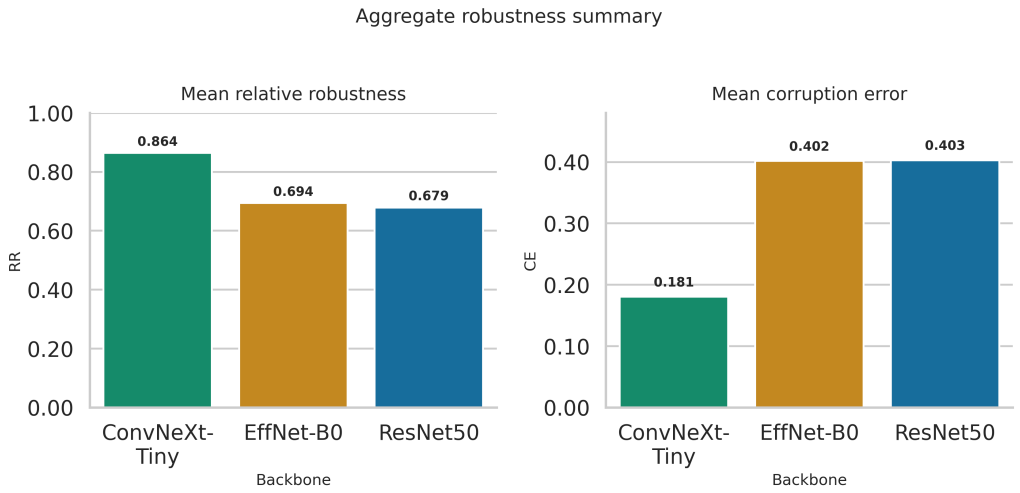

Figure 24: Aggregate robustness summary across all models and corruption conditions.

- **Clean accuracy and robustness are related, but not identical.** ConvNeXt-Tiny leads on both axes, but the ordering between ResNet50 and EfficientNet-B0 changes depending on the corruption type. This means robustness cannot be inferred from clean accuracy alone.
- **EfficientNet-B0 is only competitive under very mild corruption.** It can remain stable at low noise levels, but its performance collapses quickly as corruption severity increases, especially for blur and strong noise.
- **The practical conclusion is conservative.** ConvNeXt-Tiny is the safest default when input quality is uncertain, but none of the three models is genuinely robust to motion blur, so blur-aware augmentation or robustness-oriented training would still be necessary in deployment.

7 Layer-Wise Feature Probing (Scenario 4.5)

7.1 Protocol

This scenario examines how semantic abstraction evolves across network depth by probing features extracted from early, middle, and final convolutional stages of each frozen backbone. The backbone weights used are the original ImageNet-pretrained weights; no task-specific training of the backbone is involved.

Layer selection. Three depth levels were instrumented with forward hooks per model:

Model	Early	Middle	Final
ResNet50	`layer1`	`layer2`	`layer4`
EfficientNet-B0	`blocks.0`	`blocks.3`	`blocks.6`
ConvNeXt-Tiny	`stages.0`	`stages.1`	`stages.3`

Table 8: Layer hooks used for feature extraction at each depth level.

Feature extraction. Spatial feature maps at each hook point were reduced to a single vector per image via global average pooling, yielding one fixed-dimensional descriptor per image per layer. Features were extracted from the full 1,399-image validation set for logistic regression probing, and from a fixed subset of 30 samples per class (900 images total, same indices across all models and layers) for PCA visualization.

Linear classifier probing. A scikit-learn logistic regression with standard scaling (zero mean, unit variance) was trained on 80% of the extracted features and evaluated on the remaining 20%, using stratified splits with seed 123. This is an internal probe split over extracted features, not a separate dataset-level test split. The resulting held-out probe accuracy measures how linearly separable the frozen features are at each depth.

7.2 Results

Table 9 reports held-out probe accuracy, mean feature ℓ_2-norm, and feature dimensionality at each depth level for all three models.

7.3 Accuracy vs. Depth

Figure 25 shows probe accuracy vs. depth for all three models on a single plot. Figures 26–28 show the per-model view.

7.4 Feature Norm Statistics

Figure 29 shows the mean feature ℓ_2-norm across depth levels for all models.

7.5 PCA 2D Visualizations

Figures 30–38 show PCA 2D projections of the fixed 900-sample subset (30 classes $\times$ 30 samples) at each depth level for each model. The same sample indices are used across all models and layers.

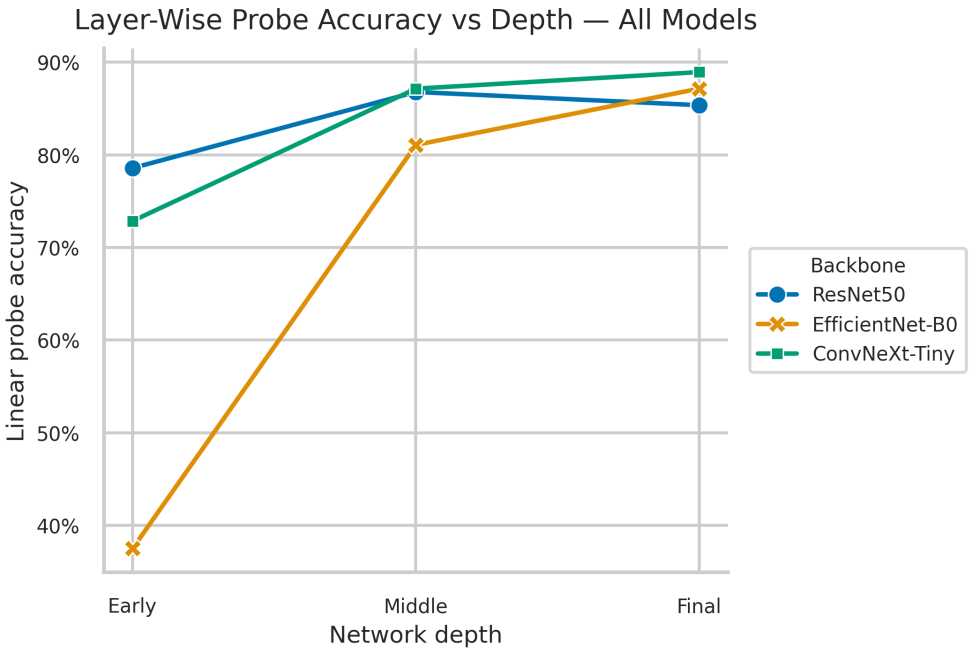

Figure 25: Layer-wise probe accuracy vs. depth for all three backbones on the same axes.

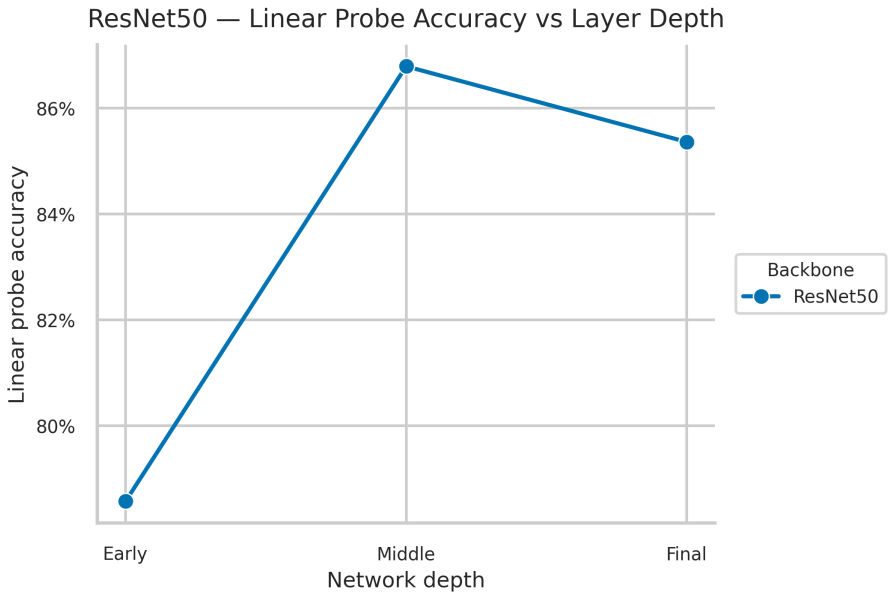

Figure 26: ResNet50: linear probe accuracy vs. layer depth.

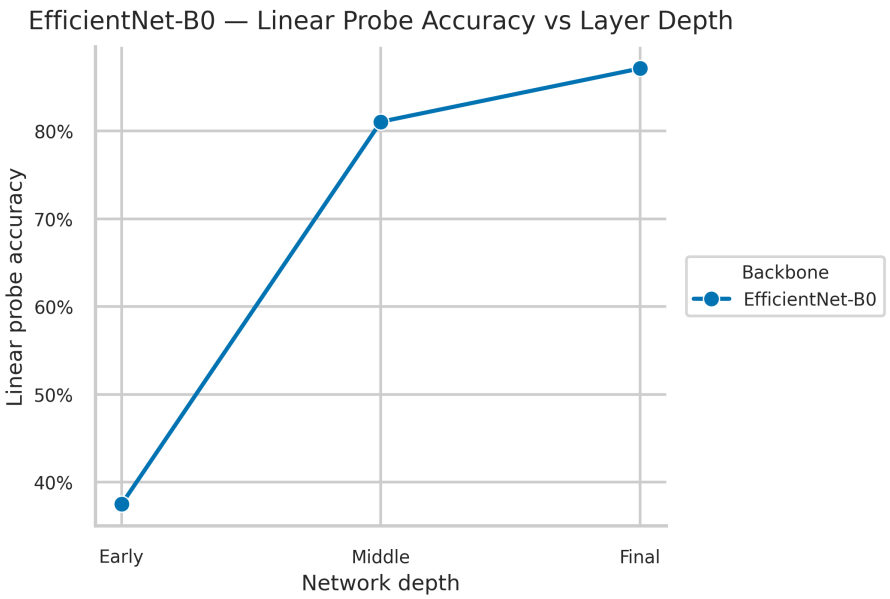

Figure 27: EfficientNet-B0: linear probe accuracy vs. layer depth.

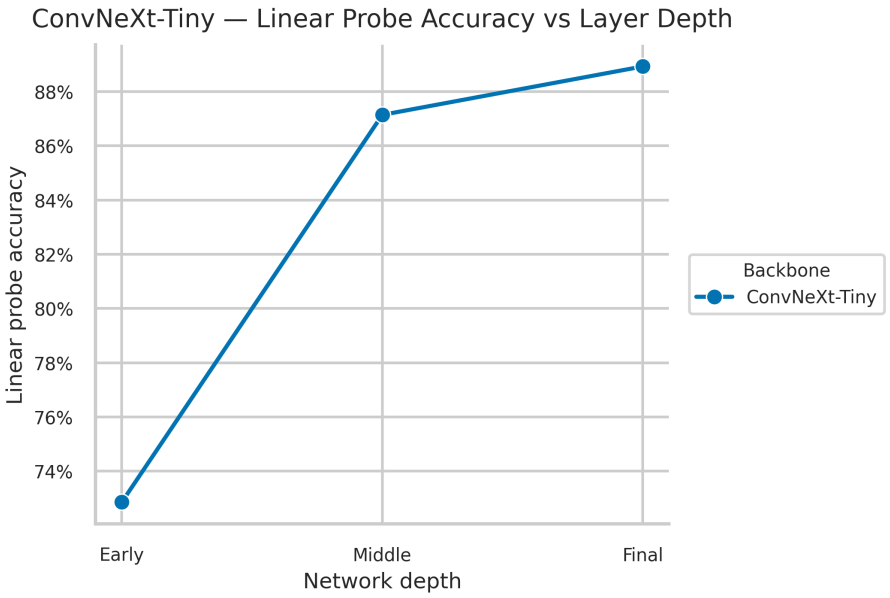

Figure 28: ConvNeXt-Tiny: linear probe accuracy vs. layer depth.

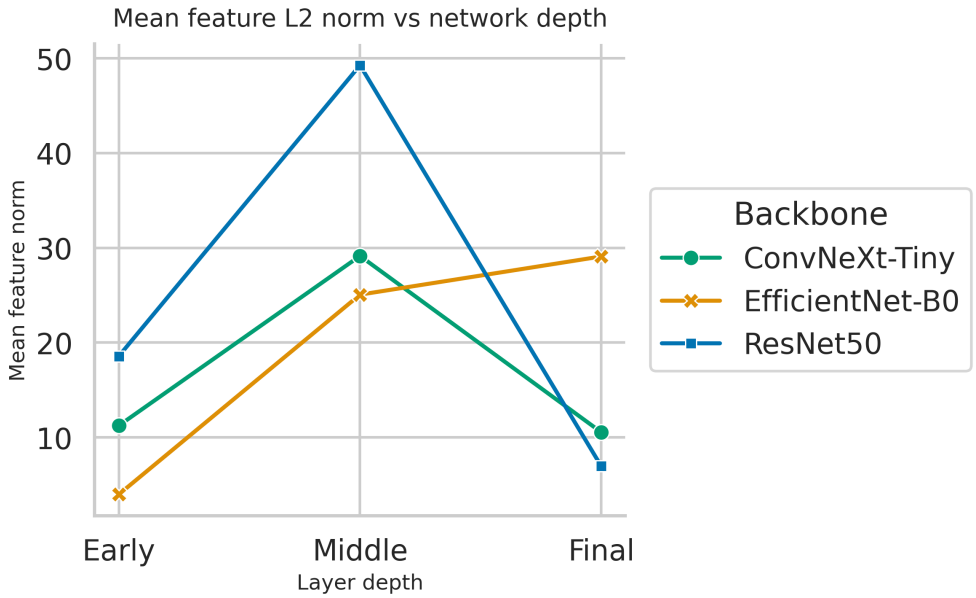

Figure 29: Mean feature ℓ_2-norm vs. layer depth for all three backbones.

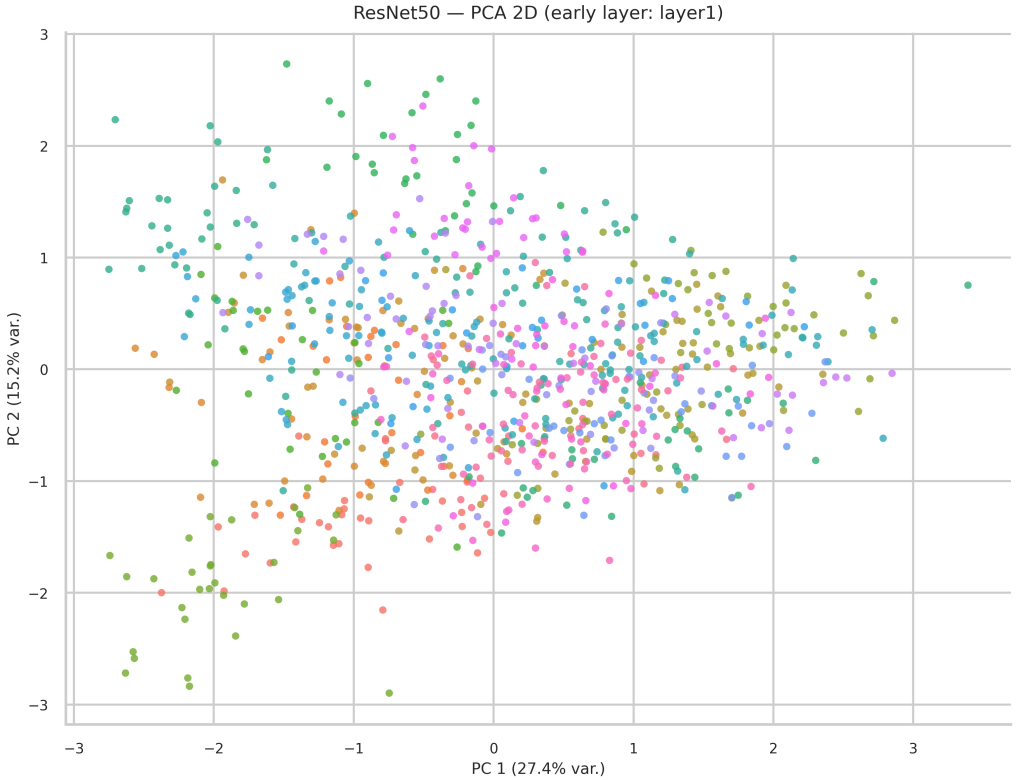

Figure 30: ResNet50, `layer1` (early): PCA 2D projection of fixed subset features.

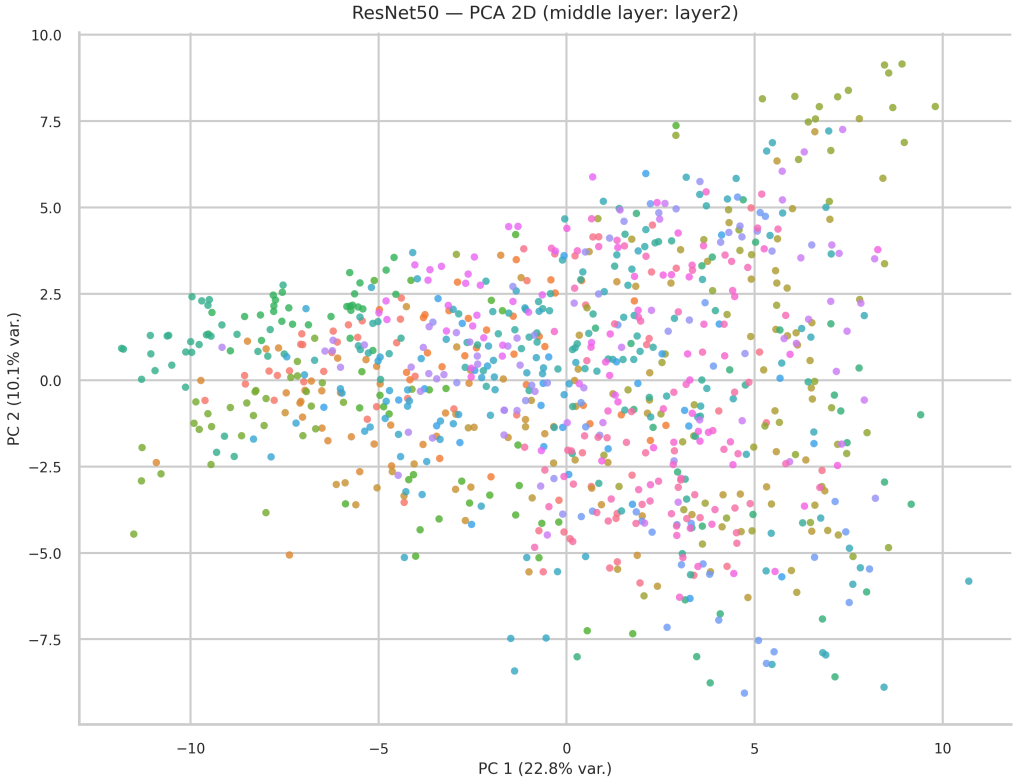

Figure 31: ResNet50, `layer2` (middle): PCA 2D projection of fixed subset features.

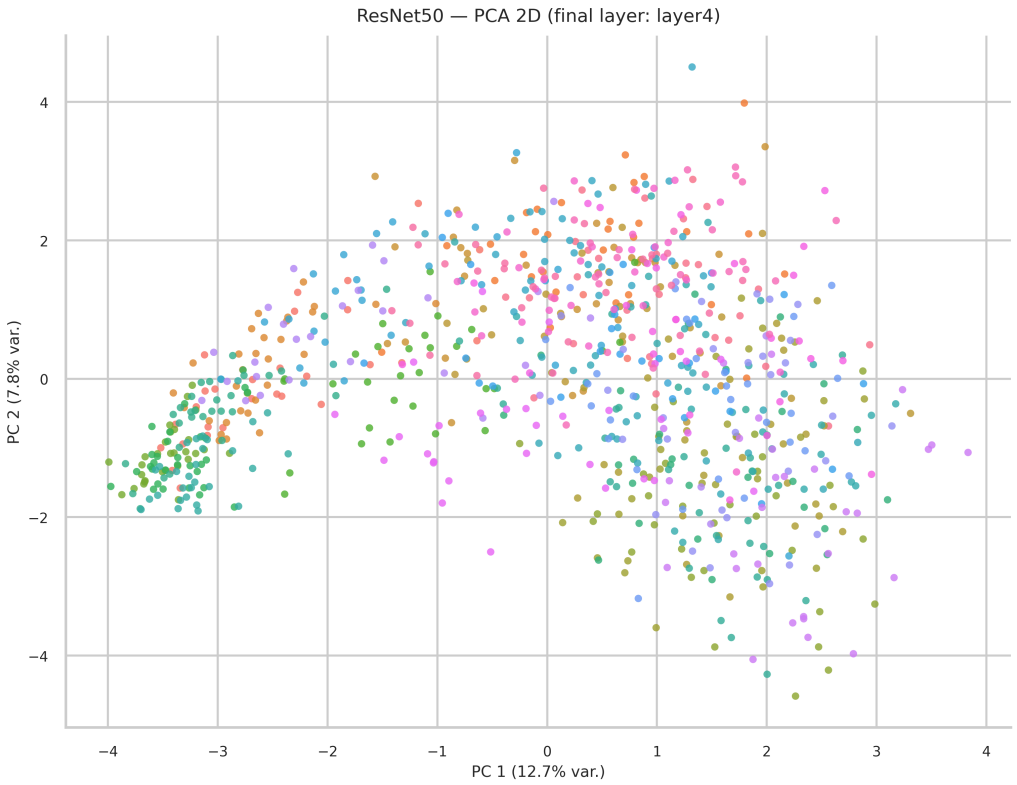

Figure 32: ResNet50, `layer4` (final): PCA 2D projection of fixed subset features.

Figure 33: EfficientNet-B0, `blocks.0` (early): PCA 2D projection of fixed subset features.

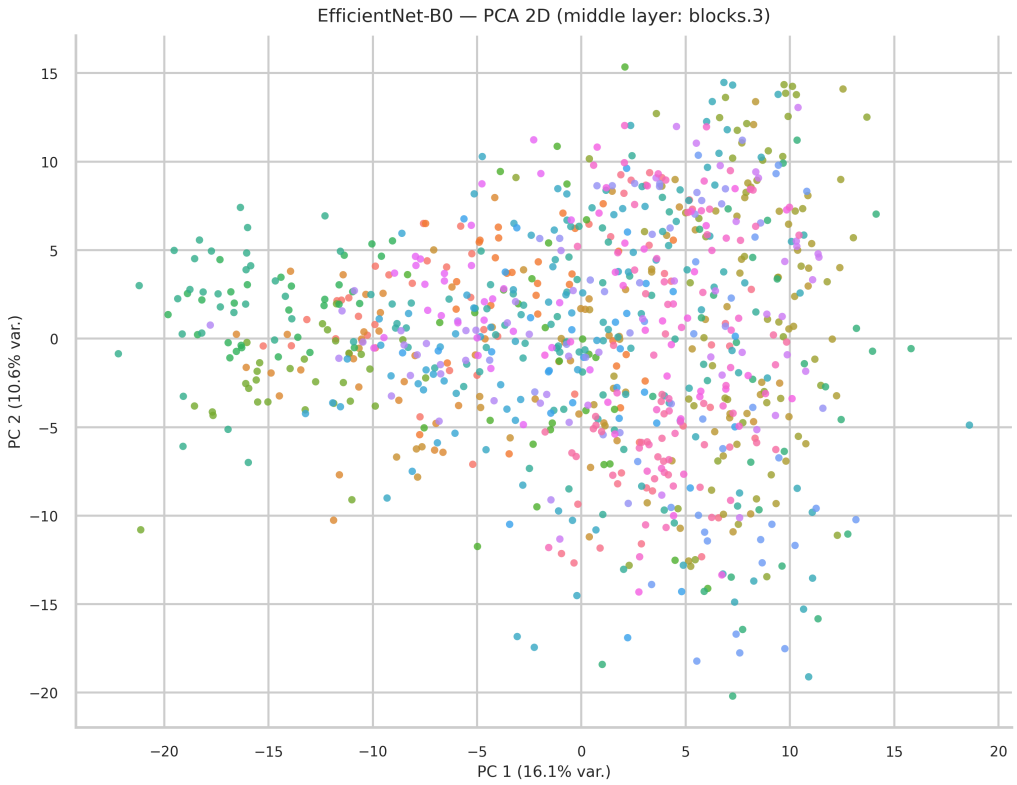

Figure 34: EfficientNet-B0, `blocks.3` (middle): PCA 2D projection of fixed subset features.

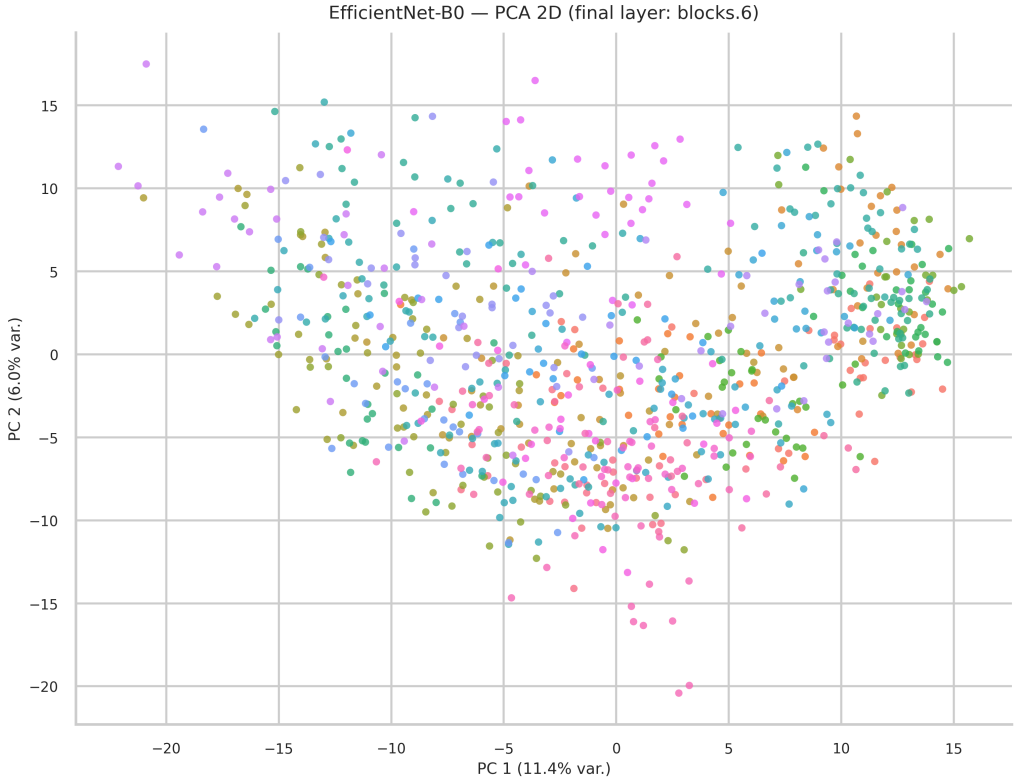

Figure 35: EfficientNet-B0, `blocks.6` (final): PCA 2D projection of fixed subset features.

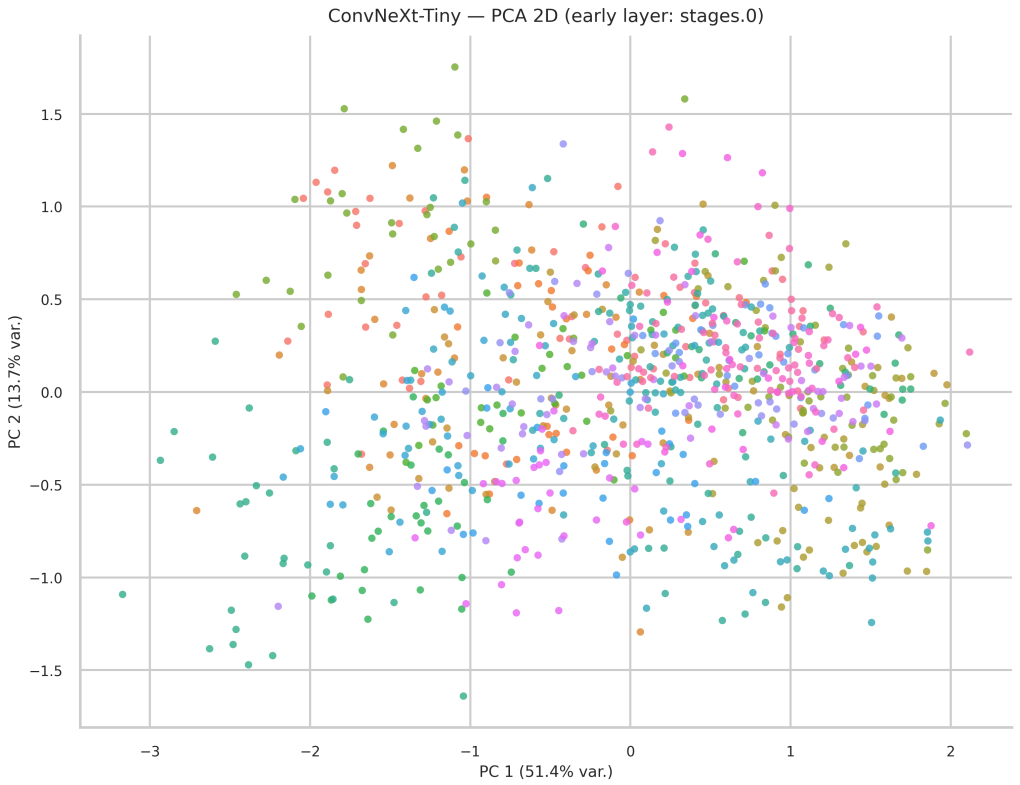

Figure 36: ConvNeXt-Tiny, `stages.0` (early): PCA 2D projection of fixed subset features.

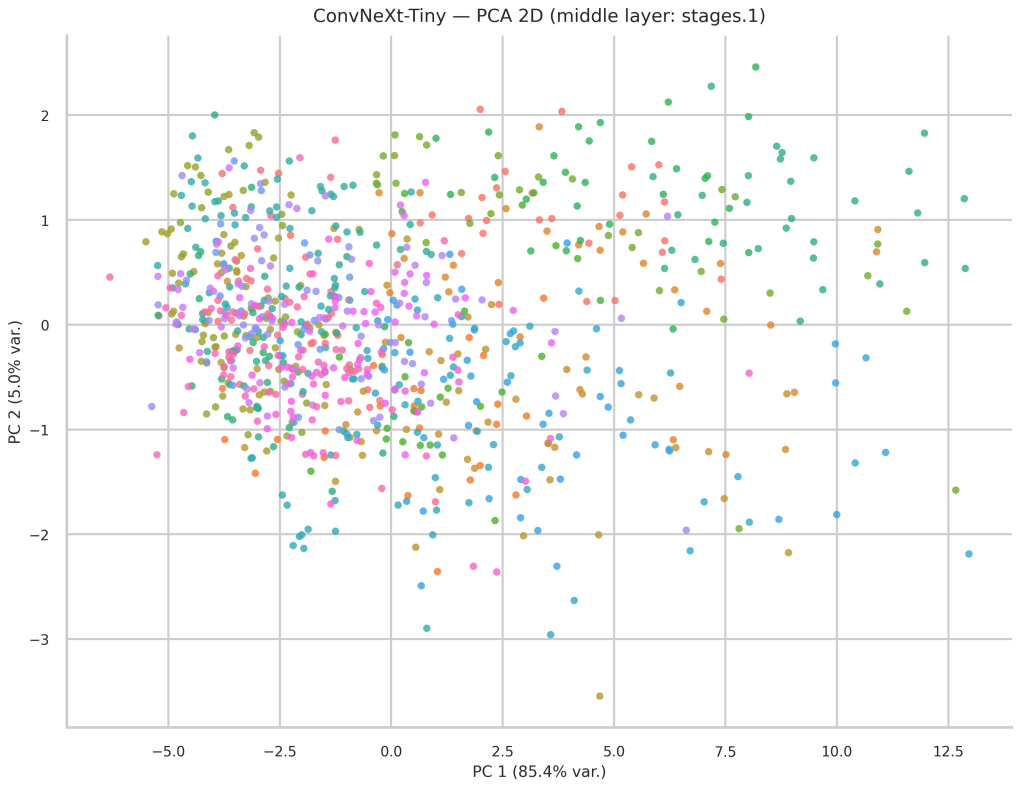

Figure 37: ConvNeXt-Tiny, `stages.1` (middle): PCA 2D projection of fixed subset features.

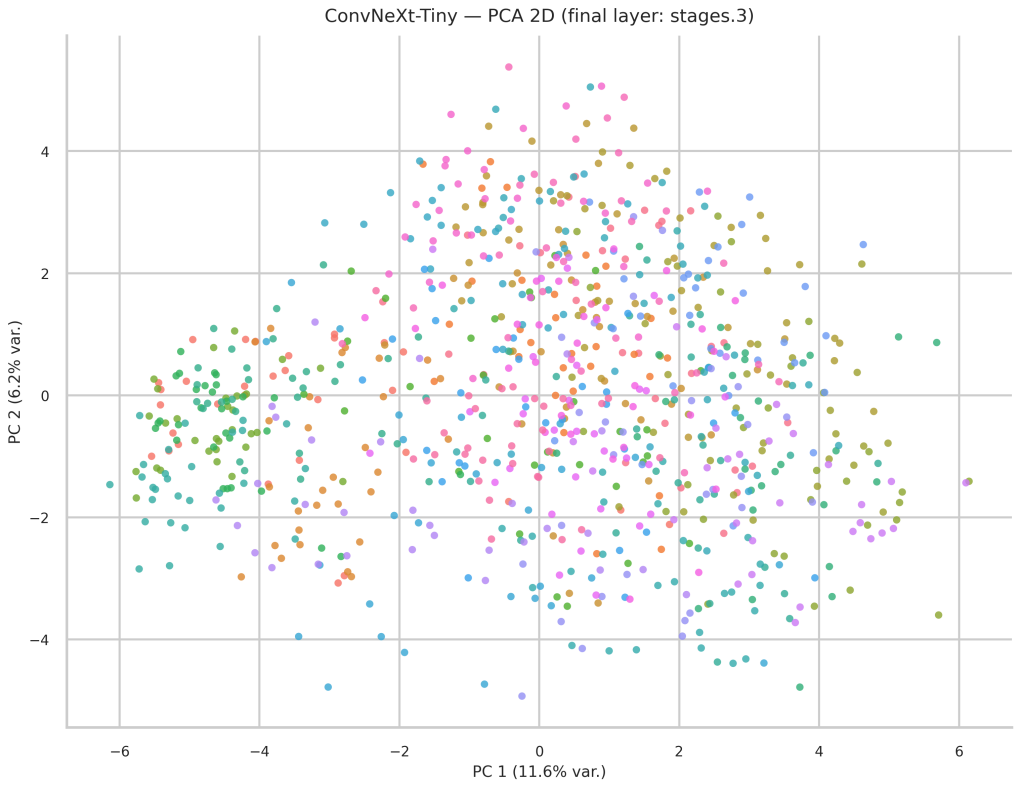

Figure 38: ConvNeXt-Tiny, `stages.3` (final): PCA 2D projection of fixed subset features.

Model	Layer	Path	Probe Acc (%)	Mean Norm	Feat Dim
ResNet50	Early	`layer1`	78.57	18.57	256
ResNet50	Middle	`layer2`	86.79	49.27	512
ResNet50	Final	`layer4`	85.36	6.96	2048
EffNet-B0	Early	`blocks.0`	37.50	3.98	16
EffNet-B0	Middle	`blocks.3`	81.07	25.06	80
EffNet-B0	Final	`blocks.6`	**87.14**	29.09	320
ConvNeXt-T	Early	`stages.0`	72.86	11.24	96
ConvNeXt-T	Middle	`stages.1`	87.14	29.15	192
ConvNeXt-T	Final	`stages.3`	**88.93**	10.54	768

Table 9: Layer-wise probing results. Probe Acc is logistic regression held-out accuracy on extracted features from the internal 80/20 probe split. Mean Norm is the average ℓ_2-norm of the GAP feature vectors.

7.6 Analysis

- **Feature quality generally improves with depth.** EfficientNet-B0 and ConvNeXt-Tiny both achieve their best probe accuracy at the final layer, showing that deeper features become more semantically useful for AID classification.
- **ResNet50 is the important exception.** Its middle layer slightly outperforms the final layer, which shows that deeper is not always better for transferability. Late-stage specialization can sometimes reduce linear separability for the target task.
- **The weakest and strongest representations are clearly separated.** EfficientNet-B0's early layer is the poorest representation in the study, while ConvNeXt-Tiny's final layer is the strongest overall.
- **PCA trends support the quantitative probe results.** Deeper layers usually produce tighter and more structured clusters, even if two-dimensional projections do not fully separate every class.
- **Feature norm magnitude should not be over-interpreted.** Better probe accuracy does not simply correspond to larger norms; what matters is class separability in the full feature space, not the raw scale of the activations.

8 Cross-Scenario Summary and Statistical Analysis

8.1 Scenario 4.1: Efficiency Trade-off

Figure 39 plots best validation accuracy against total parameter count and MACs for the linear probe setting, visualizing the accuracy-efficiency trade-off across the three backbones.

Table 10 reports best validation accuracy with 95% confidence intervals (Wilson score interval on the 1,399-image validation set) alongside efficiency metrics.

Model	Best Val Acc (%)	95% CI Low	95% CI High	Params (M)	MACs (G)	FLOPs (G)
ResNet50	87.92	86.11	89.52	23.57	4.13	8.26
EfficientNet-B0	86.20	84.29	87.91	4.05	0.39	0.78
ConvNeXt-Tiny	**94.85**	93.56	95.89	27.84	4.48	8.96

Table 10: Scenario 4.1 linear probe accuracy with 95% Wilson confidence intervals and efficiency metrics.

8.2 Scenario 4.2: Strategy Heatmap

Figure 40 shows a heatmap of best validation accuracy for every model-strategy combination, providing a compact cross-model comparison of fine-tuning strategies.

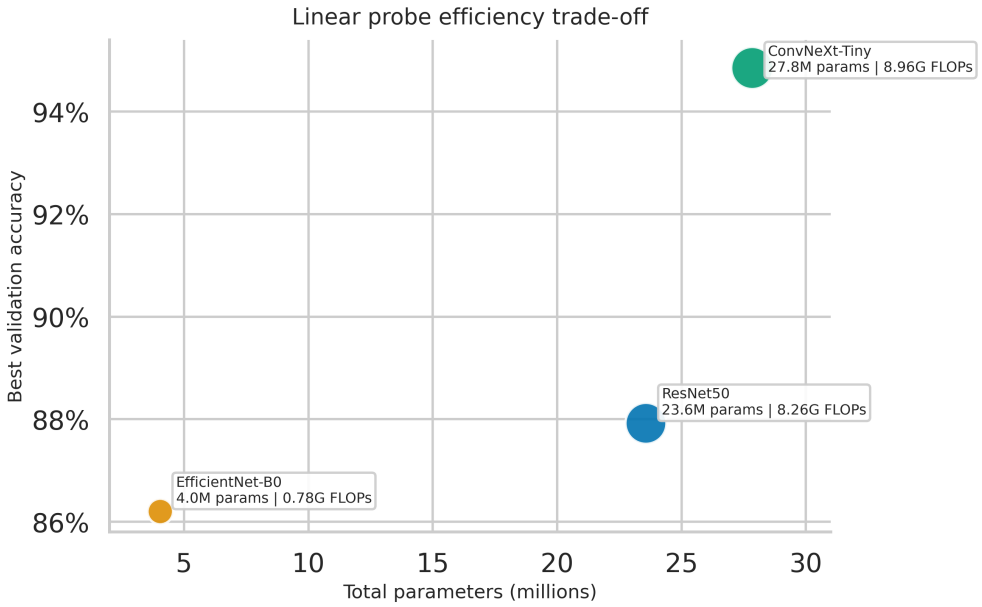

Figure 39: Scenario 4.1: accuracy vs. model complexity trade-off. EfficientNet-B0 achieves competitive accuracy at a fraction of the parameter cost; ConvNeXt-Tiny dominates in accuracy but at higher complexity.

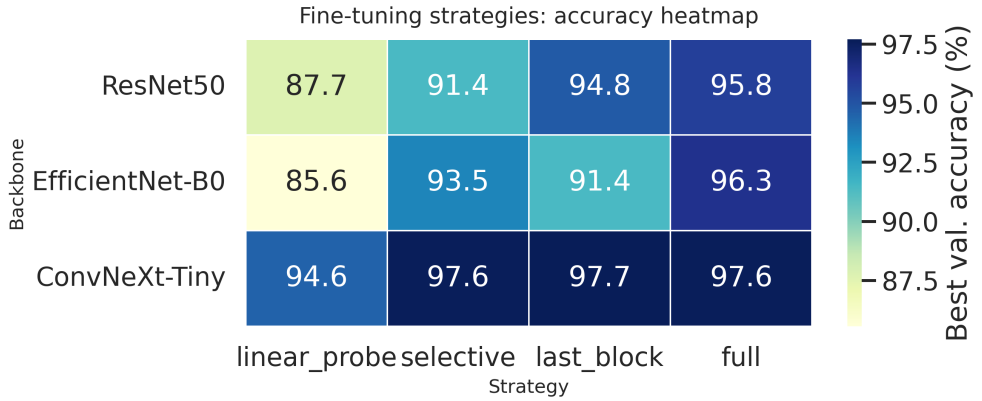

Figure 40: Scenario 4.2: heatmap of best validation accuracy (%) across models and fine-tuning strategies. Darker cells indicate higher accuracy.

8.3 Statistical Comparison

Table 11 reports pairwise statistical comparisons between models under the linear probe setting (Scenario 4.1), using a two-proportion z-test on the 1,399-image validation set.

Model A	Model B	Δ (pp)	z-score	p-value
ResNet50	EfficientNet-B0	+1.72	1.356	0.175
ResNet50	ConvNeXt-Tiny	−6.93	−6.583	4.6×10^{-11}
EfficientNet-B0	ConvNeXt-Tiny	−8.65	−7.898	2.9×10^{-15}

Table 11: Pairwise two-proportion z-tests for Scenario 4.1 (linear probe). Δ is the accuracy difference in percentage points (Model A − Model B). ConvNeXt-Tiny is significantly better than both other models ($p \ll 0.001$); the ResNet50 vs. EfficientNet-B0 difference is not statistically significant ($p = 0.175$).

Table 12 reports the same pairwise test at the 5% data regime (Scenario 4.3), where differences in data efficiency become much more pronounced.

Model A	Model B	Δ (pp)	z-score	p-value
ResNet50	EfficientNet-B0	+24.44	13.371	$< 10^{-30}$
ResNet50	ConvNeXt-Tiny	−18.80	−10.877	$< 10^{-26}$
EfficientNet-B0	ConvNeXt-Tiny	−43.24	−25.573	$< 10^{-100}$

Table 12: Pairwise two-proportion z-tests for Scenario 4.3 at 5% training data. All pairwise differences are highly significant. EfficientNet-B0 collapses most severely under data scarcity.

8.4 Robustness and Depth Summary

Table 13 summarises mean relative robustness across all corruption conditions (Scenario 4.4) and the depth-wise probe accuracy gain (Scenario 4.5) for each model.

Model	Mean RR	Gauss. RR	Min RR	Gain (pp)	Best Layer	Worst Corr.
ResNet50	0.679	0.676	0.324	+6.8	Middle	Gauss. σ=0.2
EfficientNet-B0	0.694	0.745	0.299	+49.6	Final	Motion Blur
ConvNeXt-Tiny	**0.864**	**0.950**	0.482	+16.1	Final	Motion Blur

Table 13: Robustness summary (S4.4) and depth probe gain (S4.5). RR = Relative Robustness. Gain = Final−Early probe accuracy gain (pp).

9 Computational Budget Report

All experiments were run on a single GPU. Wall-clock times were measured using Python's `time` module per epoch and summed. Scenarios 4.4 and 4.5 involve only inference and feature extraction respectively, so their runtimes are negligible. All runtimes are well within the 6-hour per-model per-scenario constraint.

9.1 Per-Model, Per-Scenario Timing

9.2 Strategy-Level Breakdown for Scenario 4.2

The total compute time for all five scenarios across all three models was **2.06 hours**, well within the constraint of 6 hours per model per scenario. The most expensive scenario was 4.2 (fine-tuning with four strategies), accounting for 69% of total runtime. Full fine-tuning of ConvNeXt-Tiny was the single most expensive run at 16.1 minutes, primarily because its larger parameter count and full gradient flow through all stages requires more computation per step than the frozen or partially unfrozen configurations.

Scenario	Model	Epochs	Time (s)	Time (h)
4.1 Linear Probe	ResNet50	30	291.6	0.081
4.1 Linear Probe	EfficientNet-B0	30	283.8	0.079
4.1 Linear Probe	ConvNeXt-Tiny	30	309.2	0.086
4.2 Fine-Tuning	ResNet50	4×30	1604.1	0.446
4.2 Fine-Tuning	EfficientNet-B0	4×30	1225.9	0.341
4.2 Fine-Tuning	ConvNeXt-Tiny	4×30	2273.9	0.632
4.3 Few-Shot	ResNet50	30+20+20	398.2	0.111
4.3 Few-Shot	EfficientNet-B0	30+20+20	382.6	0.106
4.3 Few-Shot	ConvNeXt-Tiny	30+20+20	413.4	0.115
4.4 Corruption	All 3 models	Inference	41.6	0.012
4.5 Layer Probing	All 3 models	Inference	43.9	0.012
TOTAL	**All**		**7408.3**	**2.058**

Table 14: Computational budget. Scenario 4.2 trains 4 strategies per model; the per-model time listed is the sum across all strategies. The 6-hour constraint applies per model per scenario.

Model	Strategy	Time (s)	Time (h)
ResNet50	Linear Probe	294.3	0.082
ResNet50	Last Block	296.7	0.082
ResNet50	Selective	466.9	0.130
ResNet50	Full	546.2	0.152
EfficientNet-B0	Linear Probe	283.3	0.079
EfficientNet-B0	Last Block	285.8	0.079
EfficientNet-B0	Selective	314.5	0.087
EfficientNet-B0	Full	342.3	0.095
ConvNeXt-Tiny	Linear Probe	289.4	0.080
ConvNeXt-Tiny	Last Block	305.6	0.085
ConvNeXt-Tiny	Selective	712.1	0.198
ConvNeXt-Tiny	Full	966.8	0.269

Table 15: Strategy-level wall-clock times for Scenario 4.2. The selective strategy is slower than linear probe due to the gradient calibration pass. Full fine-tuning of ConvNeXt-Tiny is the single most expensive individual run (0.269h).

10 Conclusion

This report studied transfer learning across three ImageNet-pretrained CNN backbones on the 30-class AID aerial scene classification dataset under five experimental scenarios.

ConvNeXt-Tiny is the best overall backbone. It achieves the highest accuracy in every scenario: 94.85% under linear probing, 97.64% under selective fine-tuning (matching full fine-tuning at one-fifth the parameter cost), 77.34% with only 5% of training data, and the highest mean relative robustness (0.864) under input corruptions. Its large-kernel depthwise convolutions, LayerNorm, and GELU activations yield representations that are both more transferable and more robust than the other two architectures.

EfficientNet-B0 is parameter-efficient but fragile. It achieves competitive accuracy under full data with full fine-tuning (96.35%), but collapses severely under data scarcity (34.10% at 5% data, $\Delta = 60.5\%$) and motion blur (25.73%). Its compact architecture trades representational richness for efficiency in a way that limits frozen transferability and low-data generalization.

ResNet50 is a robust baseline. It transfers well under linear probing (87.92%), degrades gracefully under data reduction, and shows an interesting non-monotonic depth pattern where middle-layer features are more discriminative than final-layer features, suggesting its final stage is over-specialized for ImageNet object categories.

Selective fine-tuning is the most parameter-efficient adaptation strategy. For ConvNeXt-Tiny, unfreezing only 19.4% of the backbone via gradient-budgeted selection recovers all the benefit of full fine-tuning. This validates the gradient-budgeted algorithm as an effective and principled method for constrained adaptation.

Negative / failure case. Motion blur is the clearest failure mode: all three models degrade sharply under horizontal blur (relative robustness 0.30–0.48) because it removes the local spatial and texture cues these CNN backbones rely on. This suggests that robust aerial deployment would benefit from blur augmentation during training or architectures with stronger global context aggregation.

Unexpected finding. ResNet50's middle layer (`layer2`) outperforms its final layer (`layer4`) in the layer-wise probing experiment (86.79% vs. 85.36%). This is counter-intuitive and suggests that ResNet50's deepest representations are more object-centric and less suitable for scene-level aerial classification than its intermediate representations, even though `layer4` has nearly 8× the feature dimensionality.

References

[1] G.-S. Xia, J. Hu, F. Hu, B. Shi, X. Bai, Y. Zhong, L. Zhang, and X. Lu, "AID: A Benchmark Data Set for Performance Evaluation of Aerial Scene Classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 55, no. 7, pp. 3965–3981, 2017. doi: 10.1109/TGRS.2017.2685945.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE CVPR*, pp. 770–778, 2016.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. ICML*, pp. 6105–6114, 2019.

[4] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE CVPR*, pp. 11976–11986, 2022.

[5] R. Wightman, "PyTorch Image Models," `https://github.com/rwightman/pytorch-image-models`, 2019.